

指标与看板 —— 概念讨论

创建日期: 2026-07-15 **修订日期:** 2026-07-16 **说明:** 本文档为一次关于指标体系、看板、TSDB、预计算与即时计算模式的完整讨论记录，用于后续方法论文档中指标与看板相关章节的参考。

一、指标定义 (Metric Definition)

核心定义

指标定义是关于**"量什么、怎么量、什么叫好"的完整规格说明。它是度量体系的设计图纸**——在测量发生之前，先回答：这个度量存在是为了回答什么问题？

构成要素

一个完备的指标定义包含以下六个维度：

维度	回答的问题	示例
度量对象	量什么属性？	"API 的端到端响应时间"
度量方法	从哪里取数、怎么算？	"从 API 网关的 access log 中取 request_time 字段"
单位与精度	数值以什么单位表达？	"毫秒，保留 1 位小数"
统计口径	原始数据如何聚合？	"P95，以 5 分钟为滚动窗口"
采集机制	数据如何获取与传输？	"OpenTelemetry HTTP server span → OTLP exporter → Prometheus"
判定标准	什么值算好、什么值算差？	"SLO: P95 < 200ms; SLA: P95 < 500ms"

关键洞察：统计口径是定义中最容易被忽视但最关键的维度。同样是"响应时间"，平均值、P50、P95、P99 回答的问题完全不同——平均值掩盖长尾，P99 暴露最差体验。**同一个原始数据源，统计口径不同，产生的是不同的指标。**

实现形式

在实际工程中，指标定义以以下几种形态落地：

(a) 代码中的 Instrument 定义

```
# OpenTelemetry 示例 — 定义一个 Histogram 类型的指标
response_time_histogram = meter.create_histogram(
    name="http.server.duration",
    description="HTTP server request duration in milliseconds",
    unit="ms",
)
```

这段代码定义了指标的**名称**、**语义**和**数据类型**（Histogram 意味着自动提供 P50/P95/P99 等多分位数）。

(b) 监控系统的 Recording Rule

```
# Prometheus recording rule -- 定义派生指标
groups:
  - name: api_slo_rules
    rules:
      - record: job:http_request_duration_seconds:p95_5m
        expr: histogram_quantile(0.95,
          rate(http_request_duration_seconds_bucket[5m]))
```

这条规则定义了：从原始 histogram 数据中，按 5 分钟窗口计算 P95，结果作为一个新的指标存储。

(c) 文档中的 SLI/SLO 规格

```
## SLI: API 可用性
- 定义：成功响应（HTTP 2xx/3xx）占全部请求的比例
- 统计窗口：30 天滚动
- 测量点：API 网关
- SLO:  $\geq 99.9\%$ 
- 错误预算：30 天内允许不可用  $\leq 43$  分钟
```

二、指标结果 (Metric Result)

核心定义

指标结果是**指标定义在特定时间点或时间窗口上的具体取值**。如果指标定义是"配方"，指标结果就是按配方做出来的"一道菜"——它有具体的时间戳、具体的数值、具体的标签维度。

数据结构

一条指标结果在存储层通常是一个带时间戳和标签的数值：

```
metric_name{label1="v1", label2="v2"} value timestamp
```

例如：

```
http_server_duration_ms{endpoint="/api/users", method="GET", region="us-east-1"} 187 1715731200
```

这表示：在时间戳 1715731200，/api/users 的 GET 请求在 us-east-1 区域的 P95 响应时间为 187ms。

关键属性

- **时效性**：指标结果是**某一时刻的快照**，它不解释趋势，只记录状态
- **多维性**：同一个指标名 + 不同标签组合 = 多条独立的时间序列
- **原始 vs 聚合**：从原始埋点数据到聚合指标结果之间存在一个计算链——raw spans → histogram buckets → P95 over 5min → stored as time-series

实现形式

指标结果一般存储在****时序数据库 (TSDB) ****中：

存储	典型场景
Prometheus TSDB	Kubernetes 生态的标准选择，本地存储 + 远程写入

存储	典型场景
InfluxDB	IoT / 实时分析场景
TimescaleDB	基于 PostgreSQL, 适合需要 SQL 查询的场景
ClickHouse	海量日志分析 + 指标聚合
VictoriaMetrics	Prometheus 兼容的高性能替代方案

存储层的核心设计决策是**分辨率 vs 保留期**的权衡：每秒采样 → 保留 7 天；每分钟聚合 → 保留 90 天；每小时聚合 → 保留 2 年。

三、指标图示 (Metric Visualization)

核心定义

指标图示是将指标结果（时间序列数据）**映射为视觉元素**，使人类能够感知模式、异常和趋势。它解决的核心问题是：**人的视觉系统善于识别形状变化，但不善于阅读数字表格。**

图示类型与适用场景

每种图示类型适合回答一类特定问题：

图示类型	回答的问题	典型形态
时序折线图	"这个指标在变化吗？趋势如何？"	横轴时间、纵轴值、一条或多条折线
单值面板 (Stat)	"当前值是多少？达标了吗？"	一个大数字 + 阈值色标 (绿/黄/红) + 迷你趋势线
热力图 (Heatmap)	"值的分布如何？长尾在哪里？"	横轴时间、纵轴值分桶、颜色深浅表示频次
仪表盘 (Gauge)	"当前值在量程中的位置？"	弧形刻度 + 指针，带阈值分区
柱状图	"不同类别之间比较？"	横轴分类、纵轴值，柱高表示大小
SLO 燃尽图	"错误预算还剩多少？消耗速度如何？"	横轴时间、纵轴剩余预算，斜线表示预算线
拓扑图	"系统的哪个组件出问题了？"	节点 = 组件，边 = 调用关系，颜色 = 健康状态

看板 (Dashboard) 的组织逻辑

看板是把多个指标（及非指标内容）组织在一个固定区域中显示的容器，其指标图示部分遵循**叙事逻辑**进行布局。一个好的看板不是面板的随意堆砌：

- 第 1 行：黄金信号（延迟、流量、错误、饱和度）—— 一眼判断系统是否健康
- 第 2 行：按服务的细粒度拆解 —— 定位是哪个服务出问题
- 第 3 行：基础设施指标 —— 排查是否是资源层面原因
- 第 4 行：业务指标 —— 确认是否影响用户体验

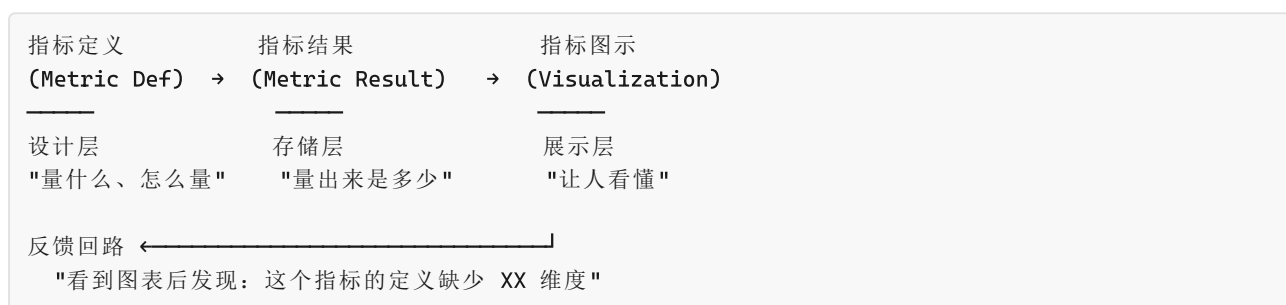
这种**从上到下、从宏观到微观**的布局，使看板成为一种**视觉诊断工具**——不需要记命令、不需要查日志，一眼定位问题层级。

实现形式

工具	特点
Grafana	开源标准方案，支持 Prometheus/InfluxDB/ES 等数十种数据源，面板类型丰富
Datadog	商业 SaaS，开箱即用的 APM + 指标 + 日志一体化
Kibana	Elastic Stack 的可视化层，偏日志分析场景
Superset	适合 BI 分析类看板，SQL 查询驱动
自研 (D3/ECharts/Recharts)	需要完全自定义交互、或嵌入产品 UI 时的选择

四、三者的关系：度量流水线

这三个概念不是孤立的，它们构成一条**度量流水线**：



反馈回路是最容易被忽视的一环：

- 你定义了一个"平均响应时间"指标
- 画成折线图看，一切正常
- 但用户投诉"有时候很卡"
- 回头看定义：平均值掩盖了长尾 → 把定义改为 P99
- 新的折线图立刻暴露了尖刺

好的度量体系不是一次定义到位的，而是在**"定义 → 观测 → 图示 → 发现盲区 → 修正定义"**的循环中持续演进的。

五、核心澄清：指标 (Metric) ≠ 单个值

"一条指标结果 = (name, labels, value, timestamp)"这个定义是**存储层的原子单元**。在实际使用中，"指标"指的是**由同一个定义产生的、共享相同 name 和 label 组合的一整条时间序列**——它是无数个单值按时间排列而成的流。

所以，准确的关系是：

指标 (Metric) = 一条时间序列 = $[v@t_1, v@t_2, v@t_3, \dots]$

指标结果 (Metric Result) = 时间序列中的一个采样点 = $v@t_n$

图示不是用一个结果画的，是用一组结果画的。不同的图示类型，区别在于"选哪些结果、怎么聚合、以什么视觉通道编码"。

六、从指标结果到图示的生成过程

这是一条查询 → 变换 → 映射的流水线：

时序数据库 (TSDB)

- ① 查询 (Query)：从 TSDB 中取出一个子集
决定：时间范围？步长/分桶？哪些 label？聚合函数？

查询结果集 (Result Set)

- 形态：一张表
- 列 = [timestamp, label₁, label₂, ..., value]
- 行 = 满足查询条件的采样点，可能几十行也可能几十万行

- ② 变换 (Transform)：在结果集上施加计算
决定：二次聚合？排序？填充缺失值？计算比率/差值？

可视化数据 (Viz Data)

- 形态：适配目标图示类型的结构
- 折线图 = [(t₁, v₁), (t₂, v₂), ...]
- 柱状图 = [(category₁, v₁), (category₂, v₂), ...]
- 仪表盘 = v_{current} (单一标量)

- ③ 映射 (Encode)：数据绑定到视觉通道
位置、长度、颜色、大小、方向 ...

图示 (Visualization)

关键环节①：查询 —— 同一条时间序列，不同“切法”产生不同图示

假设 TSDB 中存储着这样一个指标的原始数据：

```
http_duration_ms{endpoint="/api/users", method="GET"} 187 @ 10:00:00
http_duration_ms{endpoint="/api/users", method="GET"} 192 @ 10:00:15
http_duration_ms{endpoint="/api/users", method="GET"} 178 @ 10:00:30
http_duration_ms{endpoint="/api/orders", method="POST"} 512 @ 10:00:00
http_duration_ms{endpoint="/api/orders", method="POST"} 498 @ 10:00:15
http_duration_ms{endpoint="/api/orders", method="POST"} 523 @ 10:00:30
... (过去 1 小时，每 15 秒一个点，2 个 endpoint × 240 个采样 = 480 行)
```

不同图示只是对同一批数据的不同查询方式：

折线图 (趋势图)

查询：取过去 1 小时，按 1 分钟步长聚合，按 endpoint 分组

```
rate(http_duration_ms_sum[1m]) / rate(http_duration_ms_count[1m])
```

查询结果集：

```
10:01, /api/users, 185
10:01, /api/orders, 507
10:02, /api/users, 191
```

```
10:02, /api/orders, 515
...
```

映射：时间 → X 轴位置，值 → Y 轴高度，endpoint → 两条不同颜色的折线

你看到的：两条随时间波动的曲线

单值面板 (Stat)

查询：只取最近 1 分钟，取 P95，只看 /api/users

```
histogram_quantile(0.95, rate(http_duration_ms_bucket{endpoint="/api/users"}[1m]))
```

查询结果集：

```
187    （只有一个值）
```

映射：值 → 大号数字文本，值 vs 阈值 → 颜色（绿/黄/红）

你看到的：一个大数字 "187 ms"，底色绿色表示正常

柱状图

查询：只取当前时刻，按 endpoint 分组，不按时间展开

```
http_duration_ms
```

等价于问："现在这个瞬间，每个 endpoint 的延迟是多少？"

查询结果集（或取最近一个采样点）：

```
/api/users,    187
/api/orders,   512
/api/payments, 298
```

映射：endpoint → X 轴分类，值 → Y 轴柱高

你看到的：三根柱子，分别代表三个 endpoint 的当前延迟

仪表盘 (Gauge)

查询：同 Stat，取一个标量值。区别在于**映射方式不同**。

同样的单值 187：

- Stat 面板 → 显示为数字 + 颜色
- Gauge 面板 → 显示为指针在弧线上的位置，量程 0~500

你看到的：指针指在弧线约 37% 的位置，绿色区域

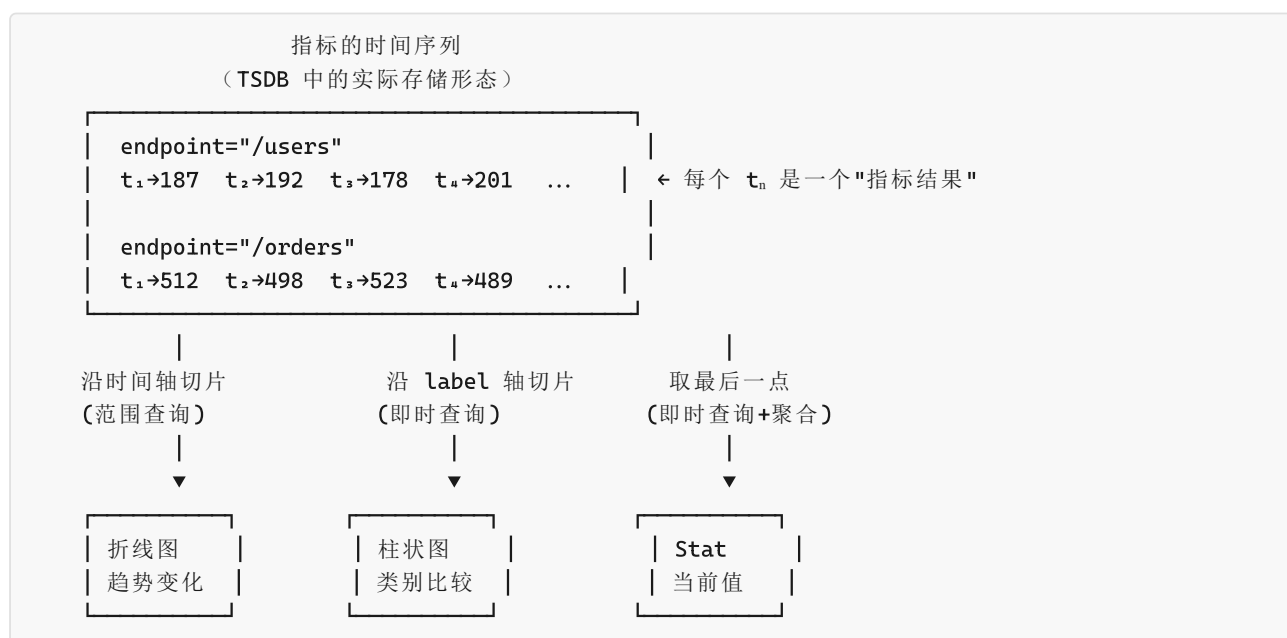
关键洞察：图示类型 = 查询维度 + 聚合方式 + 视觉通道的不同组合

图示	查询的维度	聚合方式	视觉通道
折线图	按时间展开 + 可选按 label 分组	时间窗口聚合 (rate/avg/max...)	X=时间, Y=值, 颜色=分组
柱状图	按 label 分组，不按时间展开	当前值或时间段内聚合	X=类别, Y=值

图示	查询的维度	聚合方式	视觉通道
Stat	单 label 组合, 不展开任何维度	取最后一个值或时间段聚合为标量	文本大小, 颜色
Gauge	同 Stat	同 Stat	弧线角度, 颜色分区
热力图	按时间展开 + 按值分桶	频次统计	X=时间, Y=值区间, 颜色深浅=频次
表格	按多个 label 展开	不聚合, 逐行展示	—
SLO 燃尽图	按时间展开 + label 合并	累计计算	X=时间, Y=剩余预算

所以：**所有这些图示，画的都是“同一个指标的多个结果”的某种投影。** 区别仅在于：从时间序列这个二维平面上，你切了哪一刀、从哪个角度看。

一张图总结



三个图示的数据源完全一样，但查询从 TSDB 的二维空间中取出了不同的子集、施加了不同的聚合，最终映射到不同的视觉形态。**图示不是在展示“一个指标的一个结果”，而是在展示“一个时间序列在某个维度组合下的投影”。**

七、name 是什么

name 就是这个指标“量的是什么”的标识符。 它回答了：你在看什么指标？是延迟、还是流量、还是错误数、还是 CPU 使用率？

```

http_request_duration_ms  ← 这是 name
http_requests_total       ← 这是另一个 name
cpu_usage_percent         ← 这是另一个 name

```

name 在系统中的角色：它是索引的主键。在 TSDB 中，同一个 name 的所有数据存放在一起，构成一张逻辑上的“表”。你要查什么指标，首先指定 name。

```
PromQL: http_request_duration_ms{...} ← name 写在前面
SQL:    SELECT value FROM metrics WHERE name = 'http_request_duration_ms'
```

八、label 是什么

label 是对指标数据的“定语”——同一个指标，在不同条件下有不同的值，label 就是区分这些条件的维度。

你无法只问“响应时间是多少？”这个问题本身是不完整的——你只能问“哪个接口、在哪个机房、被哪种请求方法调用的响应时间是多少？”这些“哪个”就是 label。

一个具体例子

TSDB 中存储的原始数据是这样的：

```
http_request_duration_ms{endpoint="/api/users", method="GET", region="us-east-1"} 187
http_request_duration_ms{endpoint="/api/users", method="GET", region="eu-west-2"} 245
http_request_duration_ms{endpoint="/api/orders", method="POST", region="us-east-1"} 512
http_request_duration_ms{endpoint="/api/orders", method="POST", region="eu-west-2"} 498
```

这里：

- `http_request_duration_ms` 是 **name**（四个数据点属于同一个指标）
- `endpoint`、`method`、`region` 是 **label 名**
- `"/api/users"`、`"GET"`、`"us-east-1"` 是 **label 值**

每一组不同的（`label1=值`，`label2=值`，...）的组合，形成一条**独立的时间序列**。上面 4 行 = 4 条独立的时间序列，各自随时间产生新的数据点。

label 的两个关键特征

① 基数 (Cardinality) 决定成本

label	可能的取值数
<code>region</code>	3 (<code>us-east-1</code> , <code>eu-west-2</code> , <code>ap-southeast-1</code>)
<code>endpoint</code>	~50 (系统中的 API 路径数)
<code>user_id</code>	数百万 (每个用户一个值)

label 取值数的乘积 = 时间序列总数。`region(3) × endpoint(50)` = 150 条序列，尚可管理。但如果把 `user_id` 放进 label，就会产生数百万条序列——TSDB 会被撑爆。**label 用来区分基础设施维度（机房、服务名、接口路径），不应用来区分业务实体（用户ID、订单号）。**

② Label 组合实现了“查询 = 筛选 + 分组”

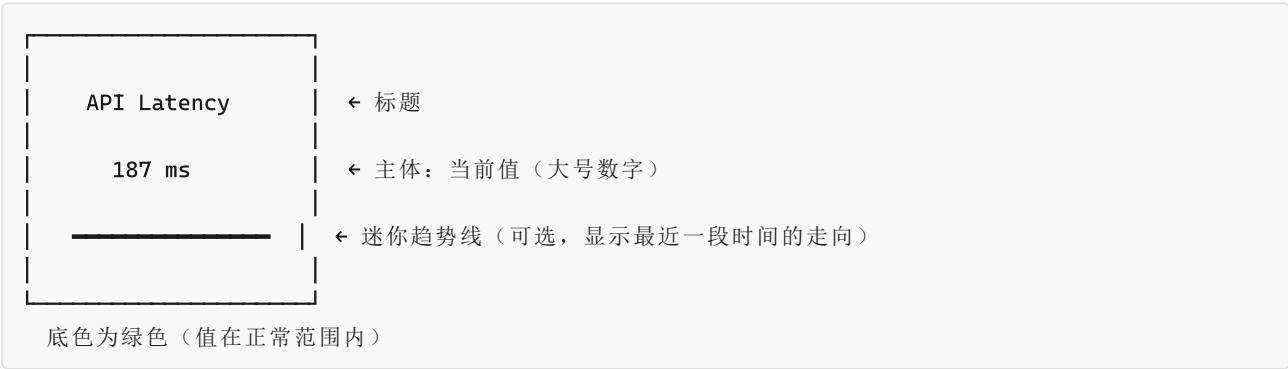
```
# 筛选：只看 us-east-1 的数据
http_request_duration_ms{region="us-east-1"}

# 分组聚合：按 endpoint 分组，计算每个 endpoint 的 P95
avg by (endpoint) (http_request_duration_ms)
```


九、Stat 是什么

Stat 是 Grafana 中的一种面板类型，用于展示"当前值是多少"。它不展示趋势、不展示分布，只展示一个数字。

Stat 的特征



Stat 的核心设计语言是：

- **一个大数字**：让你不需要细看就能感知状态。人类阅读一个数字的时间约 200ms，而阅读一张图表需要 2-5 秒。
- **阈值色标**：值 < 200 → 绿色；200 ≤ 值 < 500 → 黄色；值 ≥ 500 → 红色。颜色在视觉感知上比数字更快——你还没看清数字，颜色已经告诉你状态了。
- **可选的迷你趋势线**：在数字下方有一条压缩到一行高的小趋势图（sparkline），让你同时感知"趋势方向"。

Stat 与其他面板的区别

从"展示多少维度的信息"这个角度看：

	展示当前值	展示历史趋势	展示分布形态
Stat	✓	✗（可选迷你）	✗
仪表盘 (Gauge)	✓	✗	✗
折线图	✗	✓	✗
热力图	✗	✓	✓
表格	✓	✗	✗（但可多行对比）

Stat 适用场景：当你已经知道要关注什么，只需要确认它是否正常。比如运维大屏上一排 Stat：延迟、错误率、QPS、可用性——4 个数字，1 秒扫完，就知道系统是否健康。

Stat 不适用场景：当你需要诊断"为什么延迟突然升高"——这需要折线图来关联时间点的事件（"10:03 延迟尖刺 ← 10:03 刚好有发布"）。

十、label 与维度的关系

本质上是同一个概念，但"维度"是分析语言，"label"是实现语言。区别在于说话的语境。

分析语境（你在思考"怎么切分数据"） → 用"维度"
工程语境（你在写查询语句或埋点代码） → 用"label"

	维度 (Dimension)	Label
属于哪个层面	数据分析的概念模型	监控系统的实现机制
表达什么	"我想从哪个角度切分数据"	key="value" 键值对
使用场景	设计指标体系时, 讨论"需要哪些维度"	写 PromQL 时, 写 {region="us-east-1"}
类比	数据库表的"列"这个概念	具体某列的列名和值

一个例子说明两者的对应

分析语境 (维度) :

"API 延迟这个指标, 我需要按**接口路径**和**机房**两个维度来拆分分析。"

工程语境 (label) :

```
http_duration_ms{endpoint="/api/users", region="us-east-1"}
```

维度"接口路径"→ label 的 key 叫 `endpoint` ; 维度"机房"→ label 的 key 叫 `region` 。

同一个维度概念, 落地为不同的 label 命名——这是分析意图到工程实现的映射。

为什么 Prometheus 用"label"而不用"dimension"这个词?

这是有意的设计选择。Prometheus 的数据模型有意做得**极简**——它不区分"维度表"和"事实表", 不预先定义 Schema, 每个 label 都是扁平的 key-value。相比之下, "dimension"这个词在 OLAP 领域有更强的 Schema 预设意味 (维度通常需要预定义层次结构: 国家 → 省 → 城市)。Prometheus 用"label"这个词, 强调它只是一个**自由附加的标记**, 没有 Schema 约束。

所以: **label 就是 Prometheus 生态对"维度"的工程实现。当你讨论数据切分的思路时叫它"维度", 当你写查询或埋点时就叫它"label"。**

十一、PromQL 是什么

PromQL (Prometheus Query Language) 是 Prometheus 的查询语言, 用来从时序数据库中取出和聚合指标数据。它的定位类似于"时序数据的 SQL"。

为什么需要一种新查询语言?

时序数据的查询需求和关系型数据库不同——你不是在查"某条记录", 而是在做**时间窗口内的聚合和运算**。

SQL 擅长的: `SELECT * FROM orders WHERE user_id = 12345`
(精确查找某几行)

PromQL 擅长的: 过去 5 分钟, 每个 API 的 P95 延迟, 按每秒的速率表达
(时间窗口 + 聚合 + 速率换算 + 分位数计算)

这种需求用 SQL 当然也能写, 但会很冗长。PromQL 把"时间窗口内的聚合"变成了一等公民。

一个例子看懂 PromQL 的核心语法

假设你想看"过去 5 分钟内，每个 endpoint 的 P95 延迟":

```
histogram_quantile(0.95,
  rate(http_request_duration_seconds_bucket[5m]))
```

拆开来看：

片段	含义
<code>http_request_duration_seconds_bucket</code>	指标名称：原始延迟数据的 histogram 分桶
<code>[5m]</code>	时间窗口：过去 5 分钟
<code>rate(...)</code>	函数：把累计计数器换算为每秒增长速率
<code>histogram_quantile(0.95, ...)</code>	函数：从 histogram 桶分布中计算第 95 分位数的值

这一行表达了："在最近 5 分钟的时间窗口上，从延迟分布的 histogram 中，算出 P95 值，并换算为每秒速率。"

用 SQL 实现同样的逻辑，需要子查询、窗口函数、自定义分桶计算——远没有这么简洁。

PromQL 的定位



PromQL 是 TSDB 和图示之间的桥梁——Grafana 面板里的每一张图表，背后都是一条或多条 PromQL 查询。你配置面板时，写的就是 PromQL 表达式。

十二、TSDB 是什么

TSDB (Time Series Database, 时序数据库) 是专门为"按时间顺序到达、带时间戳的数值序列"而设计的数据库。Prometheus 自带一个 TSDB，它就是你写的 PromQL 查询的底层数据来源。

它和普通数据库有什么本质区别？

写入模式完全不同：

普通数据库 (MySQL/PostgreSQL) :

写入 = INSERT (偶尔 UPDATE / DELETE)

数据是 "记录"——订单、用户、文章, 你可以随时增删改

TSDB:

写入 = APPEND ONLY (持续追加, 几乎从不修改历史数据)

数据是 "采样"——CPU 每 15 秒采样一次, 旧的采样点永远不会被修改

这两个特征的组合——**写入压倒性地多于读取、只追加不修改**——决定了 TSDB 在内部分配、压缩、过期策略上和普通数据库走完全不同的路。

举个例子感受量的差异:

一个中等规模的微服务集群, 200 个服务实例, 每个实例暴露 50 个指标, 每 15 秒采集一次:

每秒写入: $200 \times 50 \div 15 \approx 667$ 个数据点

每分钟: $\approx 40,000$ 个

每天: $\approx 5,760$ 万个

TSDB 的设计目标就是让这种写入量保持低成本、低延迟——普通关系型数据库在这种负载下会迅速退化。

TSDB 内部怎么存数据?

两个核心设计决策:

① 按时间序列组织, 不是按时间戳组织

传统思路 (按时间排列所有数据) :

t_1 : cpu{host=a}=80, cpu{host=b}=50, mem{host=a}=60, mem{host=b}=40

t_2 : cpu{host=a}=82, cpu{host=b}=51, mem{host=a}=61, mem{host=b}=41

...

→ 同一时刻的数据写在一起, 但查 "host=a 的 CPU 历史" 需要跳着读

TSDB 思路 (每个 label 组合独立存储) :

序列 1: cpu{host=a} → [80@ t_1 , 82@ t_2 , 81@ t_3 , ...]

序列 2: cpu{host=b} → [50@ t_1 , 51@ t_2 , 52@ t_3 , ...]

序列 3: mem{host=a} → [60@ t_1 , 61@ t_2 , 59@ t_3 , ...]

...

→ 每条序列连续存储在磁盘上, 查 "host=a 的 CPU 历史" 是顺序读

这个设计的直觉是: **查询几乎总是沿着一条时间序列取一段连续的历史** ("host=a 的 CPU 过去 1 小时"), 而**不是取一个时刻的所有指标**。让这个模式高效, 就是把同一条序列的数据在磁盘上放在一起。

② 压缩策略利用了时间序列的特性

时间序列有三个利于压缩的特征:

- **时间戳是等差数列** (每 15 秒一个) → 不存绝对时间, 存 delta, 通常 0 字节
- **相邻采样的值接近** (CPU 从 80% 到 82%, 差值只有 2) → 存 delta-of-delta
- **很多值长期不变或为零** → 游程编码

效果: 一个 (timestamp, value) 对 —— 原始 16 字节 —— 压缩后通常只需 1-3 字节。

典型案例: Prometheus TSDB 的存储布局

/data/

└─ wal/

← Write-Ahead Log (写入缓冲, 防丢)

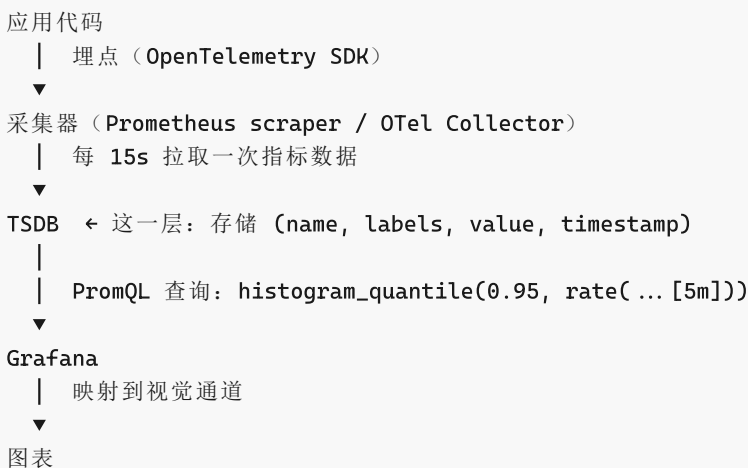


数据从**内存**（最近数据，毫秒级查询）→**WAL**（防崩溃丢失）→**Block**（每 2 小时刷到磁盘）→ 后台合并小 Block 为大 Block。旧 Block 可以被“过期删除”——超过保留期（比如 15 天）的 Block 直接删掉，这是 TSDB 的“DELETE”方式——不是逐行删，是整个文件删。

几种常见 TSDB 的对比

	Prometheus TSDB	InfluxDB	TimescaleDB	VictoriaMetrics
本质	Prometheus 内嵌的专用存储	独立的时序数据库	PostgreSQL 扩展	Prometheus 兼容的替代存储
查询语言	PromQL	InfluxQL / Flux	SQL	MetricsQL (PromQL 超集)
适合场景	K8s 监控的标准方案	IoT 传感器数据	需要时序+关系数据混合查询	Prometheus 的大规模替代（更高压缩比、更快查询）

在整条链路中的位置



TSDB 是**静默的中间层**——你很少直接和它交互，但它决定了“能查多快的数据”“数据能保留多久”“支持多少种聚合函数”。PromQL 的语法的设计本质上是在**表达“对 TSDB 中存储的时间序列施加什么操作”**。

十三、预计算路径：历史取值序列

基于指标定义，如何记录、查询和显示指标的历史取值序列——这是预计算路径的典型场景：

指标定义 (name + labels + 采集规则)

- 采集器按固定间隔采样, 每个采样点 = (timestamp, value)
- TSDB 将这些采样点按 label 组合存储为独立的时间序列
- PromQL 从时间序列中按时间窗口查询、聚合、运算
- Grafana 将查询结果映射为折线图/柱状图/Stat 等图示

这条链路的核心特征是: **指标定义是一份, 但产生的采样点是持续的、按时间排列的流**——图示本质上是在展示这条流在时间轴上的不同投影。

十四、指标"现值"的查询实现

取决于指标的类型。"现值"对于不同类型的指标, 含义完全不同, 获取方式也不同。

两种查询模式

PromQL 有两种查询模式, 这是理解"如何取现值"的基础:

即时查询 (Instant Query):

问: "现在是多少?"

返回: 一个时刻的一个/一组值

用于: Stat 面板、Gauge 面板、告警规则判断

范围查询 (Range Query):

问: "过去一段时间的每个步长是多少?"

返回: 一个时间范围内的一串值

用于: 折线图、热力图

要取"现值"走的是**即时查询**。

Gauge (仪表盘型指标) —— 直接取最后一个采样点

这是最直观的情况。Gauge 表示一个可以升也可以降的值: CPU 温度、内存使用量、当前在线用户数。

```
cpu_temperature_celsius{host="web-01"}
```

TSDB 中存储的是:

```
cpu_temperature_celsius{host="web-01"} 78.5 @ 15:02:00
cpu_temperature_celsius{host="web-01"} 78.7 @ 15:02:15
cpu_temperature_celsius{host="web-01"} 78.6 @ 15:02:30
```

即时查询返回的"现值"就是**时间戳最晚的那个采样点**——78.6。不需要计算, 只是取出来。

Counter (计数器型指标) —— 必须计算速率, 原始值本身没有意义

Counter 是一个只增不减的值: HTTP 请求总数、处理的字节数。它的原始值是累积量。

错误写法: 直接取 counter 的最后一个值

```
http_requests_total{endpoint="/api/users"} → 15,238,947
```

正确写法: 计算过去 5 分钟的每秒增长率

```
rate(http_requests_total{endpoint="/api/users"}[5m]) → 342.5
```

为什么原始值没意义？因为 15,238,947 这个数字告诉不了你"系统现在的吞吐量是多少"——它只是从进程启动以来的累计计数。你需要知道它在**最近的单位时间内增加了多少**，这才是"当前的速率"。

`rate()` 函数的计算逻辑：

```
过去 5 分钟内的 counter 增长量 ÷ 300 秒 = 每秒速率

= (15,238,947 - 15,136,147) ÷ 300
= 102,800 ÷ 300
= 342.5 请求/秒
```

这个 342.5 才是你放到 Stat 面板上的"现值"。

Histogram（直方图型指标）—— 从桶分布中计算分位数

Histogram 将数据分入预定义的取值区间（桶）。它不存每个具体值，存的是"每个桶里落入了多少样本"。

原始存储形态：

```
# 桶的累积计数
http_duration_bucket{le="50"}    → 12,340    # 延迟≤50ms的请求累积数
http_duration_bucket{le="100"}   → 23,500    # 延迟≤100ms的请求累积数
http_duration_bucket{le="200"}   → 29,800    # 延迟≤200ms的请求累积数
http_duration_bucket{le="+Inf"}  → 30,000    # 全部请求总数

# 总和与计数
http_duration_sum    → 2,850,000 # 所有请求延迟的总和（毫秒）
http_duration_count → 30,000    # 总共多少个请求
```

取"当前 P95"需要即时查询配合计算：

```
histogram_quantile(0.95,
  rate(http_duration_bucket[5m]))
```

计算过程分为两步：

第一步： `rate(... [5m])` 把每个桶的累积计数换算为最近 5 分钟内的每秒增量（理由同 Counter）。

第二步： `histogram_quantile(0.95, ...)` 在所有桶之间做线性插值——找到"95% 的请求落在哪个延迟值以下"。如果第 50ms 桶覆盖了 80% 的请求，第 100ms 桶覆盖了 96% 的请求，那么 P95 在这两个桶之间插值，大约在 93ms 左右。

最终的单个数字——比如 187 ——就是"当前 P95 延迟"。

Summary（摘要型指标）—— 客户端已算好分位数，直接取

Summary 和 Histogram 类似，但分位数计算在**采集端（应用进程内）**完成，而不是查询时计算。它直接暴露 P50、P90、P99 等预计算好的值：

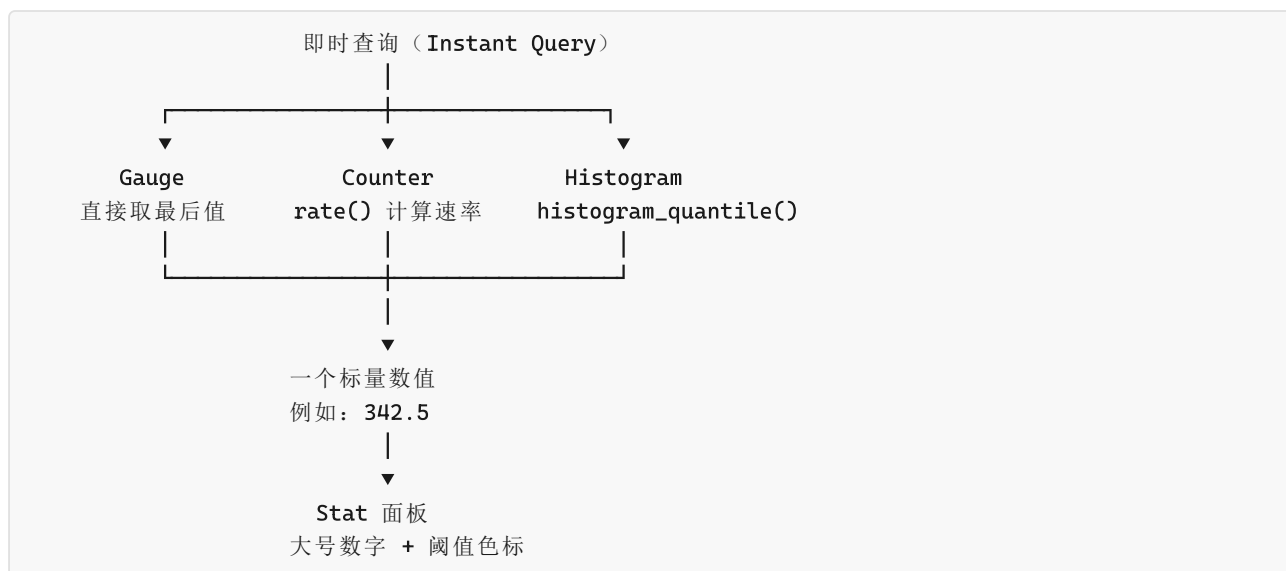
```
http_duration_quantile{quantile="0.95"} → 178
```

取"现值"就是直接取这个值，不需要 `histogram_quantile()` 函数。代价是分位数在客户端写死，事后无法换一个分位看，也不能跨服务聚合。

汇总

指标类型	"现值"是什么	查询方式
Gauge	最近一次采样的值	直接取 <code>cpu_temp</code>
Counter	最近一个时间窗口内的变化速率	<code>rate(http_requests_total[5m])</code>
Histogram	从最近窗口的桶分布中算出的分位数	<code>histogram_quantile(0.95, rate(duration_bucket[5m]))</code>
Summary	客户端预计算的分位数	直接取 <code>duration_quantile{quantile="0.95"}</code>

回到 Stat 面板



折线图用的是**范围查询**——返回一串值，连成线；Stat 用的是**即时查询**——返回一个值，显示为数字。但查询的对象、指标定义、底层 TSDB 存储是完全一样的，只是**查询模式不同**。

十五、预计算模式 vs 即时计算模式

两种范式的根本差异

"计算发生在什么时候"以及"原始数据是否保留"是两个模式的根本分水岭。

预计算路径 (Prometheus/TSDB)

业务数据 → 聚合/采样 → TSDB 存储聚合后的时间序列 → 查询时直接取

↑
原始明细被丢弃

优势: 查询快 (直接取已算好的值)

代价: 无法下钻 (原始数据已经不存在了)

只能回答 "P95 是多少", 无法回答 "哪些请求构成了这个 P95"

即时计算路径 (OLAP/BI 引擎)

业务数据 → 明细表 (每一行都保留) → 查询时现场计算 → 返回指标值

↓
点进去 → 显示原始明细

优势：可下钻、可任意维度组合、指标定义可事后调整
代价：每次查询都要扫描大量明细数据，延迟高

这两种路径不是替代关系，而是**互补关系**。

存储差异

预计算存储（TSDB）：

api_latency_p95{endpoint="/users"} 187 @ 10:05

即时计算存储（OLAP 表）：

request_id	endpoint	latency_ms	user_id	region	timestamp
req_001	/api/users	142	u_123	us-east-1	10:00:01
req_002	/api/users	198	u_456	us-east-1	10:00:02
req_003	/api/orders	523	u_123	eu-west-2	10:00:02
req_004	/api/users	167	u_789	us-east-1	10:00:03
... 数百万行					

关键差异：每一条业务事件都原样保留，不做任何预聚合。P95 不在存储时算，在查询时现场算。

十六、即时计算 + 下钻的技术实现

计算层：查询时发生了什么？

用户打开看板，看到"API 延迟 P95 = 187"。这个 187 的生成过程：

```
-- 不是 SELECT precomputed_value FROM tsdb WHERE name = 'p95'
-- 而是：

SELECT
  PERCENTILE_CONT(0.95) WITHIN GROUP (ORDER BY latency_ms) AS p95
FROM api_requests
WHERE endpoint = '/api/users'
  AND timestamp ≥ NOW() - INTERVAL '5 minutes'
```

数据库扫描过去 5 分钟内 /api/users 的全部明细行——假设 10 万行——排序，找到第 95 百分位对应的值。
每一步计算都是临时的、针对当前查询条件的。

下钻层：从聚合值回到原始行的路径

核心机制是**查询上下文传递**：

看板层

P95: 187 ms
[点击这个数字]

← Stat 面板显示的聚合值

用户点击时，前端传递：

- 指标定义：PERCENTILE_CONT(0.95) ON latency_ms
- 当前筛选条件：endpoint='/api/users'，过去 5 分钟
- 当前聚合结果：187

明细查询层

```
SELECT request_id, endpoint,
       latency_ms, user_id,
       region, timestamp
FROM api_requests
WHERE endpoint = '/api/users'
      AND timestamp ≥ NOW() - '5 min'
      AND latency_ms ≥ 180
      AND latency_ms ≤ 195
ORDER BY latency_ms DESC
LIMIT 100
```

← 继承筛选条件

← 可选：只看接近P95的
"在P95附近"的行



明细表格（按行展示原始记录）

```
req_004 | /api/users | 195 | u_999 | ... | 10:04:55
req_007 | /api/users | 193 | u_234 | ... | 10:03:12
req_012 | /api/users | 191 | u_567 | ... | 10:02:48
...
```

这个机制的本质是：**把聚合查询的上下文（指标定义 + 筛选条件 + 时间范围）原封不动转换为不带聚合函数的明细查询**。聚合查询回答了"多少"，明细查询回答了"是哪些"。

一个下钻是分层的

真正的下钻通常不止一层：

第 0 层（看板）： P95 = 187



▼ 下钻第 1 层：按 endpoint 拆分的 P95

第 1 层（分组）： /api/users P95=187

/api/orders P95=512

/api/payments P95=298



▼ 下钻第 2 层：选定 /api/orders，按 region 拆分的 P95

第 2 层（分组）： us-east-1 P95=489

eu-west-2 P95=548



▼ 下钻第 3 层：选定 eu-west-2，显示原始请求行

第 3 层（明细）： req_003 | 523 | ...

req_011 | 556 | ...

req_018 | 542 | ...



▼ 下钻第 4 层：选定 req_011，显示完整请求上下文

第 4 层（单条）： 请求体、响应体、调用链 trace、当时的环境变量

每一层的查询都是现场生成的——从上一层继承筛选条件，增加一个新的分组维度或去掉聚合函数。

实现这个架构的技术选型

即时计算 + 下钻对底层存储和计算引擎有特定的要求。核心矛盾是：**明细数据量大，但查询期望快**。

方案	计算层	存储层	即时查询延迟	下钻支持
PostgreSQL + 物化视图	PG 原生 SQL	行存	小数据量可接受，大数据量慢	原生 SQL，下钻即改写 WHERE 条件

方案	计算层	存储层	即时查询延迟	下钻支持
ClickHouse	ClickHouse SQL	列存 + 向量化	百亿行级秒出	同上，性能适合交互式下钻
DuckDB	DuckDB SQL	列存（嵌入式）	中等数据量很快	适合嵌入式/单机场景
Apache Druid	Druid SQL	列存 + 预聚合 可选	亚秒级，支持 OLAP Cube	内置 Rollup → Drilldown
StarRocks / Doris	MPP SQL	列存 + 物化视图	高并发亚秒	兼容 MySQL 协议

关键设计决策：列存 vs 行存

列存在即时聚合查询上比行存快一个数量级，因为：

行存（PostgreSQL 默认）：

查询 "P95 of latency_ms" 需要读所有列，然后只取 latency_ms 一列
→ 从磁盘读了 100MB，只用上 2MB

列存（ClickHouse）：

同一个查询只读 latency_ms 这一列的数据文件
→ 从磁盘只读 2MB

所以即时计算方案几乎总是选列存。

十七、预计算模式能否下钻看业务数据明细？

不能。预计算模式本质上是一个有损信息处理模型——“有损”正是它换取速度和低存储的代价。

预计算模式丢失了什么

预计算模式在写入 TSDB 之前，经历了一次不可逆的聚合：

业务明细（写入前）

```
req_001 | /api/users | 142ms | u_123 | us-east-1 | 10:00:01
req_002 | /api/users | 198ms | u_456 | us-east-1 | 10:00:02
req_003 | /api/orders | 523ms | u_123 | eu-west-2 | 10:00:02
req_004 | /api/users | 167ms | u_789 | us-east-1 | 10:00:03
req_005 | /api/users | 205ms | u_456 | us-east-1 | 10:00:04
... 过去 5 分钟内的 100,000 行明细
```

↓ 聚合函数（rate + histogram bucket counting）

▼

TSDB 存储（写入后）

```
http_duration_bucket{endpoint="/api/users", le="50"} 2,340
http_duration_bucket{endpoint="/api/users", le="100"} 4,500
http_duration_bucket{endpoint="/api/users", le="200"} 4,980
http_duration_sum{endpoint="/api/users"} ...
```

以及 P95 查询结果：187

从 187 这个数字出发，你无法回答：

- 哪些具体请求的延迟超过了 180ms？（re-identify 问题——聚合把个体身份消解了）
- 慢请求集中在哪个 user_id？（信息维度丢失——user_id 没有作为 label 保留）
- req_002 当时的具体请求体和响应体是什么？（payload 数据在聚合点就丢弃了）

这是数学上的不可逆，不是工程实现上的限制。 聚合函数就像哈希函数：给定 100,000 个输入值算出 P95=187，存在无限多种 100,000 个值的组合能算出同一个 P95。从结果无法唯一还原原始输入。

唯一的迂回方案：Label 维度下钻（但有硬边界）

预计算模式能做的是 **label 维度下钻**，不是业务数据明细下钻。两者的区别：

Label 维度下钻：从粗粒度 label 聚合，逐层切换到细粒度 label 聚合。

假设你在埋点时预先声明了 endpoint 和 region 作为 label：

```
看板第 1 层：  P95(全部请求) = 310
                |
                |  查询：histogram_quantile(0.95, rate(http_duration_bucket[5m]))
                |
                ▼ 下钻：按 endpoint 分组
第 2 层：      endpoint="/api/users"    P95=187
                endpoint="/api/orders"  P95=512
                endpoint="/api/payments" P95=298
                |
                ▼ 下钻：按 endpoint + region 分组
第 3 层：      /api/orders, us-east-1  P95=489
                /api/orders, eu-west-2 P95=548
```

这就是预计算模式**下钻的天花板**——你能切分的维度，仅限于埋点时声明了 label 的维度。没有声明为 label 的维度（user_id、request_id、当时的请求体内容），在聚合时已被丢弃，在查询时无法恢复。

业务数据明细下钻：从聚合值降级到原始记录行。

```
第 3 层（Label 维度下钻的终点）：
/api/orders, eu-west-2  P95=548
|
▼ 想要：看到构成这个 P95 的具体订单请求行
|
X 这在预计算模式下做不到

原因：TSDB 里存的是：
http_duration_bucket{endpoint="/api/orders", region="eu-west-2", le="500"} 3
http_duration_bucket{endpoint="/api/orders", region="eu-west-2", le="600"} 4

不是：
req_003 | /api/orders | eu-west-2 | 523ms | u_123 | ...
```

Label 维度下钻的终点仍然是聚合值，只是分组键更细了。它永远不会抵达“一行一条业务事件”的粒度。

为什么不能把所有维度都声明为 label？

这似乎是显而易见的解决方案——把 user_id 也声明为 label 不就能下钻到用户了吗？

Label 基数（Cardinality）就是硬边界。

每个 `label` 组合 = 一条独立的时间序列 = 独立的磁盘存储 + 独立的内存索引

`endpoint(50) × region(3)` = 150 条序列 ← 可以
`endpoint(50) × region(3) × user_id(10万)` = 15,000,000 条序列 ← TSDB 崩溃

TSDB 维护每条时间序列的内存索引。150 条序列占用可忽略不计的内存，但 1500 万条序列会让 Prometheus 的 RSS 膨胀到数十 GB，且任何跨序列查询都需要扫描海量索引。这不是 Prometheus 的实现缺陷，而是**预计算模型的结构性约束**——它假设序列数远小于采样点数。

所以 `label` 的选型有一个原则：**`label` 用来区分基础设施维度（机房、服务实例、API 路径），不应用来区分业务实体（用户 ID、请求 ID、会话 ID）。**

Prometheus Exemplar 的有限补救

Prometheus 在 2020 年引入了一个叫 **Exemplar** 的机制，试图在预计算模型中打一个“小洞”通回原始数据：

Histogram 桶计数	Exemplar（附加在特定桶上）
<code>le="100" → count: 4500</code>	无
<code>le="200" → count: 4980</code>	<code>trace_id="abc123", value=187</code> ← 仅附加一个样本
<code>le="500" → count: 4995</code>	无
<code>le="+Inf" → count: 5000</code>	无

Exemplar 的原理是：在 histogram 的每个时间窗口内，收集器会从落入每个桶的请求中**随机选一个**，将它的 `trace_id` 附加到桶的采样点上。当你在 Grafana 上看到 P95 异常时，可以点进去拿到这个 `trace_id`，然后跳转到 tracing 系统看那一条请求的完整调用链。

但它不是真正的下钻：

- **一个桶只保留一个样本**——你只能看到“一个例子”，看不到“所有符合条件的数据”
- **只有 histogram 类型支持**——Counter 和 Gauge 不支持
- **依赖埋点端主动发送**——需要应用代码显式附加 `trace_id`

Exemplar 的价值在于**桥接**——它不把明细存在 TSDB 里，而是存了一个指向明细存储（tracing 系统）的指针。明细本身在别处。

总结

	预计算模式	即时计算模式
原始明细	丢弃	保留
下钻深度	<code>label</code> 粒度	任意字段，直到单条记录
下钻的终点	另一个聚合值	原始业务事件行
从聚合值回到原始数据	不可能 （ Exemplar 给一个样本链接）	原生支持
查询延迟	毫秒级	秒级
存储成本	低	高（所有明细都存着）

预计算模式的下钻**不是不能，而是有硬天花板**——天花板就是 label 的基数。在这个天花板以下（按机房、服务、接口路径拆分），它可以做得很流畅；但在天花板以上（想看具体是哪个用户、哪条请求），它无能为力。

两种模式不是替代关系，是互补关系。预计算回答"系统是否健康、哪里出问题了"，即时计算回答"这个问题的根因是什么、具体影响哪些业务对象"。实际工程中，两者的衔接点——从预计算指标的异常值，跳到即时计算的明细查询——就是指标体系必须设计好的人机接口。

十八、数据量问题：即时计算意味着存储灾难吗？

先看一组真实数字

假设一个中等规模的业务系统：

每天 1000 万次 API 请求
每条请求记录 20 个字段（endpoint, latency, user_id, region, status_code ...）
原始 JSON 一行约 500 字节

天量： $1000\text{万} \times 500\text{B} = 5\text{GB}/\text{天}$
月量： $5\text{GB} \times 30 = 150\text{GB}$
年量： $150\text{GB} \times 12 = 1.8\text{TB}$

1.8TB 听起来不小。但这是**原始文本格式**的体积。列存压缩之后，数字会完全不同。

列存的压缩效率：比直觉低一个数量级

列存的核心优势不是"把同一列放在一起方便查询"——磁盘顺序读已经很快了。列存的真正威力在于**压缩**。

为什么列存压缩率极高

① 同列的值域极窄 → 字典编码

endpoint 列（50 个不同取值，1000 万行）：

行存：每行存完整字符串 `"/api/users/123/profile"` $\approx 25 \text{ 字节} \times 1000\text{万} = 250\text{MB}$

列存：建立字典 `{0:"/api/users", 1:"/api/orders", 2:"/api/payments", ...}`
每行只存一个整数 `0~49` $\approx 6 \text{ bits} \times 1000\text{万} = 7.5\text{MB}$
压缩比 $\approx 33:1$

② 排序列的 delta 编码

timestamp 列（按时间插入，天然有序）：

不压缩：每行 8 字节 $\times 1000\text{万} = 80\text{MB}$
delta编码：存相邻差值。每秒1000个请求 → 差值通常是 0ms 或 1ms
平均每个差值 $< 1 \text{ bit} \rightarrow$ 约 1.2MB
压缩比 $\approx 66:1$

③ 低基数列的游程编码

status_code 列（大部分是 200）：

实际存储："连续 5,321,847 个 200"（一个 (count, value) 对 = 几个字节）
而不是：200, 200, 200, 200, ... 重复 500 万次

综合效果

原始文本（用于对比）： 5GB/天
行存（PostgreSQL 不压缩）： ~3GB/天
列存 + 压缩（ClickHouse）： ~200MB/天 ← 通常是原始体积的 4% ~ 8%

按这个比例重新计算：

月量：200MB × 30 = 6GB
年量：200MB × 365 = 73GB

一块 1TB 的 SSD 不到 500 元。存一年的明细数据，存储成本约 40 元。

换句话说：数据量大不是问题，行存才是问题。

分层保留策略：没有人"全存永久"

即时计算 ≠ 全量永久保留。实际系统用分层保留策略：

热层（Hot）
├ 存储：内存 + NVMe SSD
├ 保留：最近 3~7 天
├ 粒度：原始明细，逐行可查
├ 用途：实时看板、异常排查、下钻明细
└ 数据量：约 1.4GB（7天）

温层（Warm）
├ 存储：SSD / HDD
├ 保留：7 天 ~ 3 个月
├ 粒度：按小时预聚合 + 1% 采样明细
├ 用途：趋势分析、周期性对比
└ 数据量：约 10GB（3个月）

冷层（Cold）
├ 存储：对象存储（S3/OSS），查询时按需加载
├ 保留：3 个月 ~ 2 年
├ 粒度：按天预聚合，无明细
├ 用途：年度报告、合规审计
└ 数据量：约 10GB（2年）

关键是：90% 的排查需求发生在最近 24 小时内，95% 在 7 天内。分层保留策略正是利用了这个时间局部性——热层保留完整明细以支持下钻，温层和冷层逐步降精度。

十九、物化视图（Materialized View）

什么是物化视图

物化视图是"把查询结果提前算好并存下来"的数据库对象。"物化"（materialize）这个词是相对普通视图的"虚拟"而言的——普通视图每次被查询时重新算一遍，物化视图直接读已算好的结果。

普通视图 vs 物化视图

普通视图（View）

定义: CREATE VIEW AS SELECT ... (只存了这条 SQL 文本)
查询: 每次 SELECT * FROM view → 现场执行底层的 SELECT → 返回结果
代价: 查询时 CPU + IO
数据: 永远和基表一致

物化视图 (Materialized View)

定义: CREATE MATERIALIZED VIEW AS SELECT ... (存了 SQL + 查询结果)
创建: 执行一次底层 SELECT → 把结果写入磁盘
查询: 每次 SELECT * FROM mv → 直接读磁盘上存好的结果
代价: 创建/刷新时 CPU + IO, 查询几乎零开销
数据: 和基表之间有时间差 (上次刷新之后基表的变更不反映在 MV 中)

本质就是“用空间 (存一份结果) 换时间 (查询不用算了)。”

一个具体例子

基表是 API 请求明细:

```
-- 基表: 每一行是一条 API 请求
CREATE TABLE api_requests (
  request_id String,
  endpoint   String,
  latency_ms Int32,
  region     String,
  status_code Int16,
  timestamp  DateTime
);
```

普通视图只是存了一条 SQL:

```
CREATE VIEW api_p95_view AS
SELECT
  endpoint,
  region,
  toStartOfMinute(timestamp) AS minute_bucket,
  quantile(0.95)(latency_ms) AS p95,
  count() AS request_count
FROM api_requests
GROUP BY endpoint, region, minute_bucket;
```

每次 Grafana 刷新面板, 数据库都要扫描原始明细表、现场排序、计算分位数。10 亿行数据, 这个查询可能需要 5-10 秒。

物化视图则是把上面这个查询的**结果**写下来:

```
CREATE MATERIALIZED VIEW api_p95_mv
ENGINE = AggregatingMergeTree()
ORDER BY (endpoint, region, minute_bucket)
AS
SELECT
  endpoint,
  region,
  toStartOfMinute(timestamp) AS minute_bucket,
  quantileState(0.95)(latency_ms) AS p95_state, -- 存中间态, 不是最终值
  countState() AS count_state
```



```
FROM api_requests
GROUP BY endpoint, region, minute_bucket;
```

创建完成后，磁盘上多了一张物理表：

endpoint	region	minute_bucket	p95	count
/api/users	us-east-1	10:00:00	187	12,340
/api/users	us-east-1	10:01:00	191	12,510
/api/users	eu-west-2	10:00:00	245	8,920
/api/orders	us-east-1	10:00:00	512	3,400
...				

Grafana 查询 `SELECT * FROM api_p95_mv WHERE ...` ——直接读这张小表，毫秒级返回。因为聚合工作在数据写入时就已经分摊完成了。

物化视图的刷新问题

核心代价：**基表有新数据写入时，物化视图不会自动同步**（手工刷新）或**有延迟地同步**（定时刷新）。

两种模式：

刷新模式	行为	适用场景
手动刷新	必须显式执行 <code>REFRESH MATERIALIZED VIEW</code>	日报、周报等非实时场景
定时刷新	数据库按固定调度（如每 5 分钟）自动刷新	看板，接受几分钟延迟
增量刷新	只刷新基表中新增/变更的部分	大数据量场景，仅部分引擎支持

增量刷新是最优策略，但需要基表有明确的增量标记（如自增 ID 或时间戳），且聚合逻辑需要支持可合并的中间态。

ClickHouse 的特殊实现：不仅存结果，而且自动增量刷新

ClickHouse 的物化视图和传统数据库有一个关键差异——它是**触发器模型**而非**定时刷新模型**：

传统数据库物化视图：

基表变更 → 什么都不发生 → 定时任务触发 → 全量/增量刷新 MV

ClickHouse 物化视图：

基表插入一个 Block → 自动触发 → 该 Block 的数据经过聚合 SQL
→ 以中间态写入 MV 的内部表 → 查询时合并中间态得到最终结果

这就是为什么前面的示例中用了 `quantileState` 而不是 `quantile`：

基表插入：10:01 这一分钟内 /api/users 的 500 个请求延迟值

|

▼ 自动触发

物化视图：把这 500 个值计算为“可合并的中间态” → 追加到 MV 表

|

▼ 查询时

`SELECT quantileMerge(p95_state)` ← 合并所有中间态 → 最终 P95

没有定时任务，没有手工刷新。基表每次插入，物化视图增补一点中间结果。查询时现场合并——但合并的代价比扫描原始明细小几个数量级，因为中间态数据量只有原始数据量的几分之一。

物化视图的表里每行是一个 (endpoint, region, minute_bucket) 组合的中间态，一小时内也就几百行。查询扫描几百行合并出一个数字——毫秒级完成。

物化视图在架构中的角色

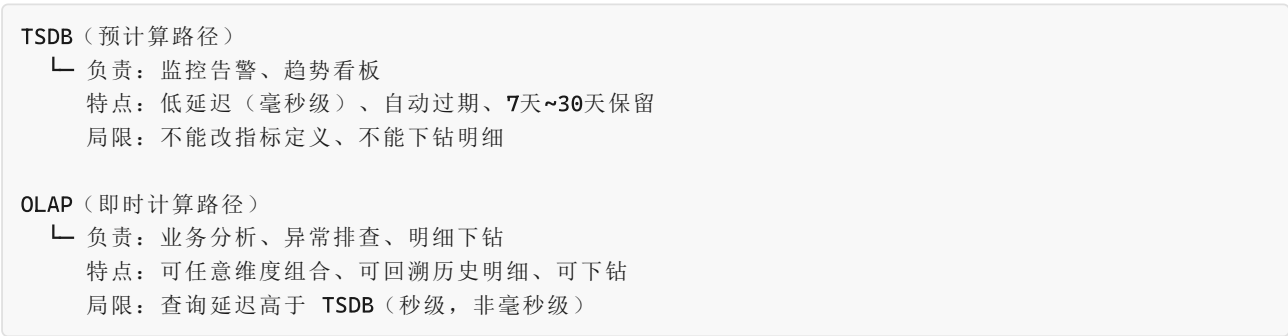
回到即时计算 + 下钻的看板场景：



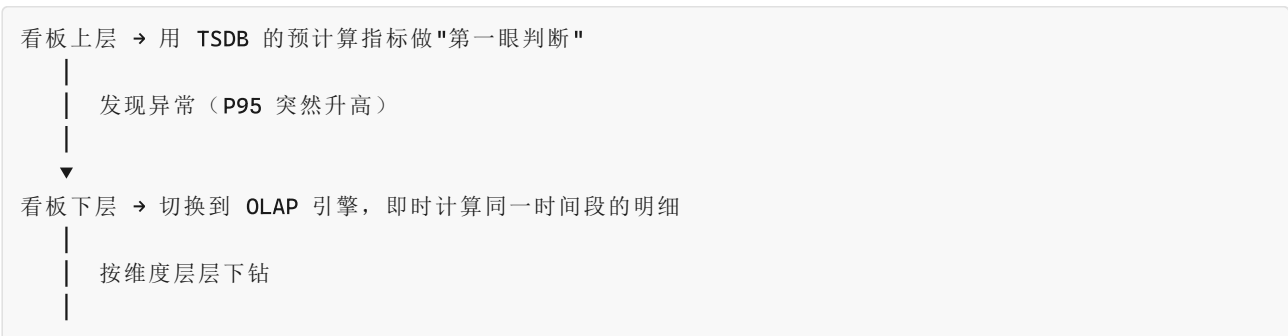
物化视图在这里解决了"既要即时计算（保留明细以支持下钻），又不想每次都扫描明细（看板加载太慢）"的矛盾。它处于**预计算和即时计算的交汇点**——写入时预计算，查询时零开销；但基表明细仍在，随时可以绕过物化视图直接查明细。

二十、两种范式在系统中的互补关系

"先看聚合指标，再下钻到业务数据明细"——在实际系统中通常是**两种范式互补使用**：



常见的集成模式：



定位到具体请求 → 跳转日志平台看调用链

二十一、关键结论汇总

维度	预计算模式	即时计算模式
原始明细	丢弃	保留
下钻深度	label 粒度	任意字段，直到单条记录
下钻的终点	另一个聚合值	原始业务事件行
从聚合值回到原始数据	不可能 (Exemplar 给一个样本链接)	原生支持
查询延迟	毫秒级	秒级
存储成本	低	中等 (列存压缩 + 分层保留)

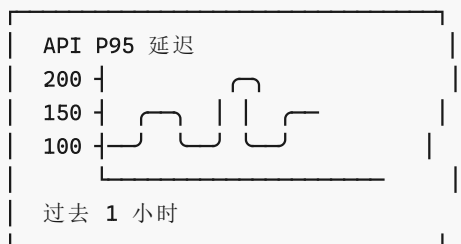
核心洞察：预计算模式是一个有损信息处理模型——“损失”是指标定义之外的、未声明为 label 的全部原始维度信息。这种“有损”不是缺陷，而是工程权衡的结果——用信息完整性换取查询速度和存储效率。理解两种模式的有损/无损边界，是做指标体系设计的前提。

二十二、如何区分指标图示来自预计算还是即时计算

从观众的视角：仅看图是区分不出来的

同一个折线图，背后可能是两种完全不同的数据路径：

你看到的 Grafana 面板：



可能的数据来源 A（预计算）：

Grafana → PromQL → TSDB → 15 秒前算好的 P95 快照

可能的数据来源 B（即时计算）：

Grafana → SQL → ClickHouse → 刚才扫描 50 万行明细现场算出 P95

折线本身不会告诉你它是哪种路径产出的。 视觉形式与数据来源是正交的——这是有意为之，因为看板的观众通常不需要关心数据怎么来的，只需要知道“这个数字是否可信、是否够快”。

但有些特征可以从侧面推断：

线索 1：时间粒度

	预计算	即时计算
最小时间步长	受采集间隔限制（通常是 15s 或 30s 的整数倍）	理论上可以是任意粒度，1 秒甚至事件级
数据点密度	均匀，固定步长	可均匀也可不均匀
"现在"这个点的含义	最近一次采样的值（可能是 14 秒前的）	截止到当前的实时计算值

如果你看到折线图最后一个点的时间戳是 10:05:00，而现在时间是 10:05:12 ——几乎肯定是预计算（采集间隔 15s，下一次采集还没发生）。如果最后一个点就是"此刻"，大概率是即时计算。

线索 2：查询延迟感知

预计算：面板刷新几乎是瞬间的（读已算好的值，毫秒级）
即时计算：面板刷新有可感知的停顿（扫描明细现场算，秒级）

但这条线索不可靠——刷新慢也可能是网络延迟或查询写得差。

线索 3：下钻能力

这是最可靠的区分线索：

点击图表上的一个数据点：

- 弹出了一个明细表，列出原始请求行
↓
背后是即时计算（保留了原始明细）
- 弹出了另一个图表，只是换了分组维度（如从 P95 总量 → 按 endpoint 拆分 P95）
↓
背后是预计算（只能做到 label 维度的细分，不能抵达记录级）
- 点击没有任何反应
↓
预计算且没有配置下钻链接

线索 4：Grafana 数据源

在面板的编辑模式里可以直接看到数据源类型：

数据源 = Prometheus → 几乎一定是预计算
数据源 = ClickHouse / PostgreSQL / MySQL → 有可能是即时计算（也可能是预置的汇总表）
数据源 = InfluxDB → 两种都可能

观众所需要知道的核心信息：下钻终点

即时计算和预计算都可以下钻，但终点不同：

即时计算 → 下钻终点 = 业务数据（一行一条订单/请求/用户操作）
预计算 → 下钻终点 = 结果明细（按 label 拆分的统计值，粒度到头了）

观众需要知道终点在哪，才不会产生"为什么点不到原始数据"的困惑。

一个字段就够了——下钻终点：

取值	观众看到	点下去是什么
业务数据	可下钻至业务明细	原始的订单/请求/操作记录，一行一条
统计结果	可下钻至分组统计	按某维度（接口/机房/时间）拆分的聚合值
外部链路	可跳转至详情系统	跳转到日志平台 / tracing 系统查看单个 trace
无下钻	不可下钻	点击无反应

面板上的表达：

API P95 延迟

187 ms

下钻至：统计结果

← 一行足够

订单处理量

3,421/min

下钻至：业务数据

不需要时效性、数据源、计算模式——那些是实现细节。观众只关心“我能往下走到哪”。

实现层面：数据源 → 元信息自动推导

实现者不需要每个面板手动配置元信息。**数据源类型直接决定了元信息：**

数据源 =	下钻终点 =	元信息 =
Prometheus	统计结果	"下钻至：统计结果"
ClickHouse 明细表	业务数据	"下钻至：业务数据"
ClickHouse 物化视图	统计结果*	"下钻至：统计结果"
PostgreSQL 明细表	业务数据	"下钻至：业务数据"

* 物化视图虽然底层是 ClickHouse，但它存的是聚合中间态。点下去如果要看业务数据，需要额外配置一条跳转到明细表的查询。

实现约定：**选好数据源 → 元信息自动确定**。面板 Description 里填一行“下钻至：业务数据 / 统计结果”，数据源选择后就确定了。

两种计算模式共存的看板：数据源层约定

一个看板里有预计算面板也有即时计算面板，核心原则：**用数据源名称区分，而非在查询里硬编码**。

看板变量（Dashboard Variables）：

\$prometheus	= Prometheus	← 所有预计算指标用这个
\$clickhouse	= ClickHouse	← 所有即时计算指标用这个
\$mv	= ClickHouse (MV)	← 物化视图用这个（可选）

面板查询：

```
# 预计算面板 -- 数据源选 $prometheus
histogram_quantile(0.95, rate(http_duration_bucket[5m]))

# 即时计算面板 -- 数据源选 $clickhouse
SELECT quantile(0.95)(latency_ms) FROM api_requests
WHERE timestamp ≥ NOW() - INTERVAL 5 MINUTE
AND endpoint = '$endpoint'
```

然后约定一个数据源 → 元信息的映射表，放在看板的说明区域：

本看板数据源说明：

数据源	下钻终点	备注
Prometheus	统计结果	按 label 维度分组下钻
ClickHouse	业务数据	可下钻至原始请求明细
ClickHouse (MV)	统计结果*	点开切换后到 raw 数据源查询明细

汇总：一个可实现的最小方案

看板层面

看板首页：数据源映射表（全局说明）

- Prometheus → 下钻至：统计结果
- ClickHouse (raw) → 下钻至：业务数据
- ClickHouse (MV) → 下钻至：统计结果*

面板层面

预计算面板

API P95 延迟 187 ms

下钻至：统计结果

[点击 → 按 endpoint 分组展开 P95 折线图]

即时计算面板

订单处理量（近1小时） 3,421/min

下钻至：业务数据

[点击 → 弹出原始订单明细表]

实际落地只需要做三件事：

1. 定义“下钻终点”的四种取值（业务数据 / 统计结果 / 外部链路 / 无下钻），沉淀为约定
2. 在面板 Description 字段填入一行“下钻至：X”
3. 看板页头放数据源映射表，让实现者选数据源时就知道下行终点应该填什么

二十三、预计算指标结果的存放与阈值设定

预计算指标结果其实已经存在了

预计算路径下，指标结果已经以时间序列的形式存在 TSDB 里了：

```
Prometheus TSDB:
  api_latency_p95{endpoint="/users", region="us-east-1"} 187 @ 10:05:00
  api_latency_p95{endpoint="/users", region="us-east-1"} 191 @ 10:10:00
  ...
```

TSDB 只存了**值本身**，不存这个值与目标值/警戒值/挑战值的关系。阈值信息和结果是分离的。需要补齐的是一套阈值存储和查询机制，让一次查询能同时拿到当前值 + 三级阈值。

阈值的存放：独立阈值表

阈值从指标定义表中独立出来。基础模型如下，完整模型（含组织×时间×指标三维立方体）见§二十五 `kpi_targets` 表：

```
-- 基础阈值模型（适用于按维度+时间范围的简单阈值管理）
-- 完整模型见§二十五 kpi_targets 表，支持组织×时间×指标三维立方体
CREATE TABLE metric_thresholds (
  threshold_id      STRING PRIMARY KEY,
  metric_id         STRING NOT NULL,          -- FK → metric_definitions

  -- 适用范围
  dimension_filter  JSON,                    -- {"endpoint": "/api/users"}, NULL = 通用
  effective_from    DATE NOT NULL,
  effective_to      DATE,                    -- NULL = 当前有效

  -- 三级阈值
  threshold_type    STRING NOT NULL,         -- "target" / "warning" / "challenge"
  comparison        STRING NOT NULL,         -- "≤" / "≥"
  threshold_value   FLOAT NOT NULL,

  -- 审计
  created_at        TIMESTAMP,
  created_by        STRING
);
```

关键设计：

- 阈值随时间演变可追溯——`effective_from` / `effective_to` 形成时间线，不 UPDATE 旧记录
- 阈值可按维度差异化——同一指标对不同 endpoint 有不同标准
- 变更历史不丢——每次调整 INSERT 新行，旧行作为历史归档
- **组织/人员维度的阈值管理**使用 `kpi_targets` 表（§二十五），通过 FK 关联 `org_nodes` 和 `time_periods`，支持年度规划、继承分解、多级阈值线叠加

结果快照表：把 TSDB 快照和阈值评价一起固化

TSDB 数据有保留期限限制。如需长期保留指标值与阈值的对比历史（如年度达标率报告），需定期快照：

```
CREATE TABLE metric_snapshots (
  snapshot_id      STRING PRIMARY KEY,
  metric_id        STRING NOT NULL,
  snapshot_time     TIMESTAMP NOT NULL,
  time_window      STRING,                  -- "1h" / "1d"
```

```

-- 实测值
current_value      FLOAT,

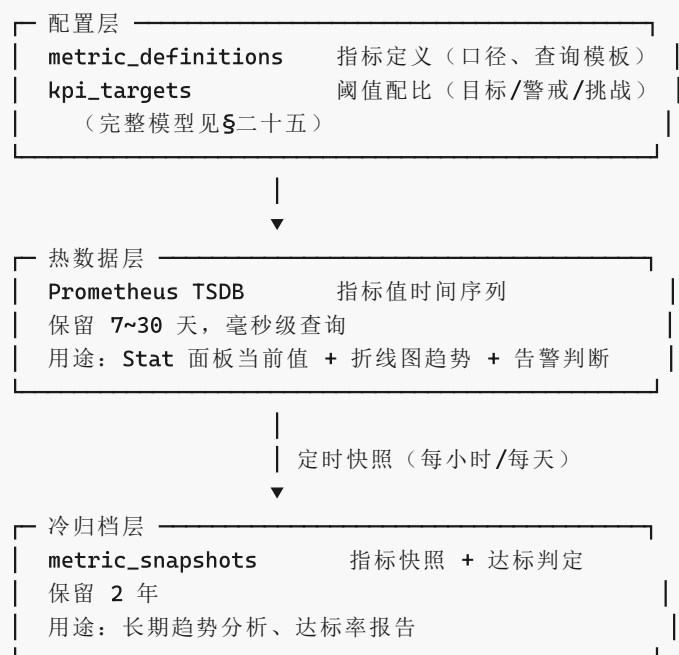
-- 维度标签
dimension_labels   JSON,

-- 冗余存储当时的阈值 + 达标判定（固化，不受后续阈值变更影响）
target_value       FLOAT,
target_comparison  STRING,
target_met          BOOLEAN,
warning_value       FLOAT,
warning_comparison STRING,
warning_triggered   BOOLEAN,
challenge_value     FLOAT,
challenge_comparison STRING,
challenge_met        BOOLEAN,

calculated_at      TIMESTAMP DEFAULT NOW(),
INDEX idx_snapshot_metric_time (metric_id, snapshot_time)
);

```

分层存储架构



二十四、阈值管理界面：让业务人员管理指标定义与目标值

核心矛盾

业务人员的语言：“订单处理的响应速度，Q3 要提高标准”。物理表的语言：向 `kpi_targets` 插入新的目标值记录。管理界面的职责就是消除这个翻译成本。

页面 1：指标目录（首页）

列头筛选模式——和 Excel 一样，每列自带筛选器和排序：

指标目录

[列表视图] [图示视图]

已筛选：4 个指标 | [业务域=支付 ×] [所属组织≡支付部 ×]

指标名称	业务域	所属组织	管理团队	周期	当前值	状态	宽度		
▼	▼▼	▼▼	▼	▼▼	▼	▼▼	▼		
支付成功	支付	支付部	支付团队	2026Q2	98.7%	▲	1/3		
订单处理	订单	支付部, 物流	订单团队	2026Q2	3421	●	2/3		
API P95	技术	支付部, 技术	平台团队	2026Q2	187ms	●	1/3		
错误率	基础	全公司	平台团队	2026Q2	0.12%	●	3/3		

* 列头上 ▼ = 有筛选器，▼ = 已启用筛选，▲/▼ = 排序方向

* 点击 × 删除筛选条件，[清除全部] 重置

[新建指标] [批量调整阈值] [导出报告]

每列的筛选器类型：

列	数据特征	筛选器
指标名称	文本	关键字搜索（包含/精确/开头）
业务域	枚举（5~10 个值）	多选勾选列表
所属组织	数组字段（组织树）	多选树形勾选 + 匹配模式（包含任一/精确）
管理团队	枚举（5~15 个值）	多选勾选列表
负责人	枚举（人名列表）	多选勾选 + 搜索
指标类型	gauge / counter / histogram	多选勾选列表
周期	年→季→月三层结构	多选（树形）
当前值	数值 + 单位	大于/小于/介于
状态	达标/未达标/警戒	多选勾选列表
图示宽度	1/3, 2/3, 3/3	多选勾选列表

排序： 点击列头文字切换升序 ▲ → 降序 ▼ → 无。支持多列排序（先点 A 列再点 B 列 = 先按 A 排，A 相同按 B 排）。

默认筛选： 用户打开页面时，系统根据角色自动填入：

KPI 主管

→ 所属组织 = "全公司"（即不筛选） + 周期 = 当前季度

部门主管

→ 所属组织 = 自己的部门 + 周期 = 当前季度

岗位角色

→ 所属组织 = 自己的岗位 + 周期 = 当前季度

所以支付总监打开页面默认看到 4 个相关指标，而不是全量 34 个。切换所属组织筛选或清空即可看其他部门的指标。

切换到图示视图：所有筛选和排序状态保持不变。图示视图不再显示表格，只显示筛选条件汇总行（可删除条件）+ 图表卡片网格。

说明：此页面基于**指标定义清单**（一行一个指标），组织/周期是筛选条件而非行维度。选择"支付部 + 2026Q2"后，列表展示 owner_org_node_ids 包含"支付部"的指标。"部门目标"列从 kpi_targets 查询，无记录则 fallback 到公司级。切换到图示视图后每个指标一张趋势图，图上叠加部门目标线和公司基准线。

页面 2：指标详情页

点击"支付成功率"进入，展示：基本定义（口径、计算公式、数据来源、维度）；当前有效阈值（目标/警戒/挑战值 + 达标判定）；达标趋势折线图（叠加三条阈值线）；阈值变更历史（谁在什么时候改了什么、为什么）。

页面 3：阈值调整

调整阈值：支付成功率

目标值

≥ 不低于 ▼

[99.7]

%

警戒值

≤ 不高于 ▼

[98.0]

%

挑战值

≥ 不低于 ▼

[99.95]

%

生效日期：

[2026-07-01]

调整原因：

[Q3 业务目标：配合新支付渠道上线 ...]

影响预览

△ 目标值从 99.5% → 99.7%:

基于近 30 天数据，达标天数将从 28 天降为 12 天

[提交审批]

[保存草稿]

关键设计：比较方式让业务人员选自然语言（"不低于"、"不高于"）；右侧建议栏基于历史数据做校准；影响预览在提交前就告诉业务人员新目标的可达性；调整原因必填——这是阈值变更的上下文记录。

页面 4：新建指标（引导式表单）

新建指标

第 1 步：基本信息

指标名称：

[支付成功率]

口径说明：

[支付接口返回成功的请求占全部 ...]

第 2 步：指标所属

业务域：

[支付服务 ▼]

所属组织（可多选）：

☒ 支付部

☐ 物流部

☐ 技术部

☒ 支付总监

☐ 物流总监

☐ 技术副总

☐ 全公司（所有组织可见）

管理团队：[支付团队 ▼]

负责人： [zhangsan ▼]

第 3 步：计算规则

度量对象：[支付请求]

度量属性：[成功率]

聚合方式：[百分比]▼

统计窗口：[5 分钟]▼

系统推断

推荐数据源：ClickHouse

推荐表：payment_gateway_logs

推荐字段：status_code, timestamp

[确认] [手动指定]

第 4 步：阈值设置

目标值：[不低于 ▼] [99.5] [%]

警戒值：[不高于 ▼] [98.0] [%]

挑战值：[不低于 ▼] [99.9] [%]

第 5 步：下钻 + 图示

下钻终点：[业务数据 ▼]

下钻明细表：payment_gateway_logs

展示字段：[order_id] [amount] [status_code] ...

默认图示宽度：[1/3 ▼]

[提交 → 自动生成 query_template 插入 metric_definitions]

关键设计：业务人员填"度量对象+度量属性+聚合方式"，系统自动翻译成 query_template。所属组织的选择决定了哪些部门/岗位的指标目录中展示此指标。所属四字段（业务域、所属组织、管理团队、负责人）共同构成指标在列表视图中的筛选维度，也是列头筛选器的数据来源。

二十五、年度 KPI 规划：目标值 × 时间 × 组织 × 人员

从零散调整到年度规划

KPI 管理部门的需求是**年初把全年各层级的目标一次性铺开**，不是事后逐条调整。这需要目标值管理的三个维度同时展开：时间（年/季/月）、组织（公司/部门/岗位）、阈值类型（目标/警戒/挑战）。

数据模型

组织维度用显式组织树，目标挂在岗位（角色）上而非自然人上——人员变动不改变目标定义：

```
CREATE TABLE org_nodes (
  node_id          STRING PRIMARY KEY,      -- "dept_payment" / "role_vp_tech"
  node_name        STRING NOT NULL,
  node_type        STRING NOT NULL,         -- "company" / "department" / "role"
```

```

    parent_id      STRING,
    sort_order     INT DEFAULT 0
);

```

时间维度用年→季→月的三层结构：

```

CREATE TABLE time_periods (
    period_id      STRING PRIMARY KEY,      -- "2026" / "2026Q1" / "2026M01"
    period_type    STRING NOT NULL,        -- "year" / "quarter" / "month"
    parent_id      STRING,                 -- "2026Q1" → "2026"
    start_date     DATE NOT NULL,
    end_date       DATE NOT NULL
);

```

目标值立方体：

```

CREATE TABLE kpi_targets (
    target_id      STRING PRIMARY KEY,
    metric_id      STRING NOT NULL,        -- FK → metric_definitions
    org_node_id    STRING NOT NULL,        -- FK → org_nodes
    period_id      STRING NOT NULL,        -- FK → time_periods
    threshold_type  STRING NOT NULL,        -- "target" / "warning" / "challenge"
    comparison     STRING NOT NULL,
    threshold_value FLOAT NOT NULL,
    source         STRING NOT NULL,        -- "manual" / "decomposed" / "inherited"
    change_reason  STRING,
    created_at     TIMESTAMP,
    created_by     STRING,
    UNIQUE (metric_id, org_node_id, period_id, threshold_type)
);

```

继承与分解规则

时间维度向下分解：年→季→月默认等值继承（“成功率 ≥ 99.5%”这类比率型指标全周期约束）。对于累积型指标（如订单处理量），支持按比例分解（考虑季节性）。

组织维度向下继承：公司设定后，自动继承到各部门、各岗位（标记 source = “inherited”）。继承的值是默认值，主管可在继承后单独修改某部门的值，修改后 source 变为 “manual”，不再受上级变更的级联影响。

查询优先级：岗位级 > 部门级 > 公司级，同精度取最新 effective_from。

KPI 主管的年度规划工作台

KPI 年度规划

规划年度：[2026 ▼] [从上年复制]

进度概览

公司级已设定		12/12	● 完成
部门级已设定		8/12	▲ 需补充
岗位级已设定		5/12	▲ 需补充
月度分解完成		3/12	▲ 待分解

[设定公司级目标] [设定部门级目标] [月度分解] [导出责任书]

公司级目标设定：网格编辑——行为指标、列为年/Q1/Q2/Q3/Q4，默认等值继承年度值，主管只修改差异格。支持全局上浮 5% 一键批量调整。累积型指标可展开月度细分面板。

部门级目标设定：每个部门默认继承公司基准，只有在有差异的地方覆盖。"公司基准"和"部门目标"并排展示，"差异"列高亮。一个部门的设置可批量复制到其他部门。"全部继承公司级"、"全部上浮 5%"、"仅调整差异项"三个快捷操作覆盖大部分场景。

岗位级：结构同部门级，组织树上呈现副总师、总监等岗位节点，默认继承所属部门。

部门/领导的查看页面

部门主管视图：概览卡片（当前值 + 达标状态 + 目标值，颜色做第一时间判断）；目标矩阵（部门目标 vs 公司基准并排，显示差异量）；月度达标趋势图（叠加部门目标线和公司基准线）；导出目标责任书 PDF。

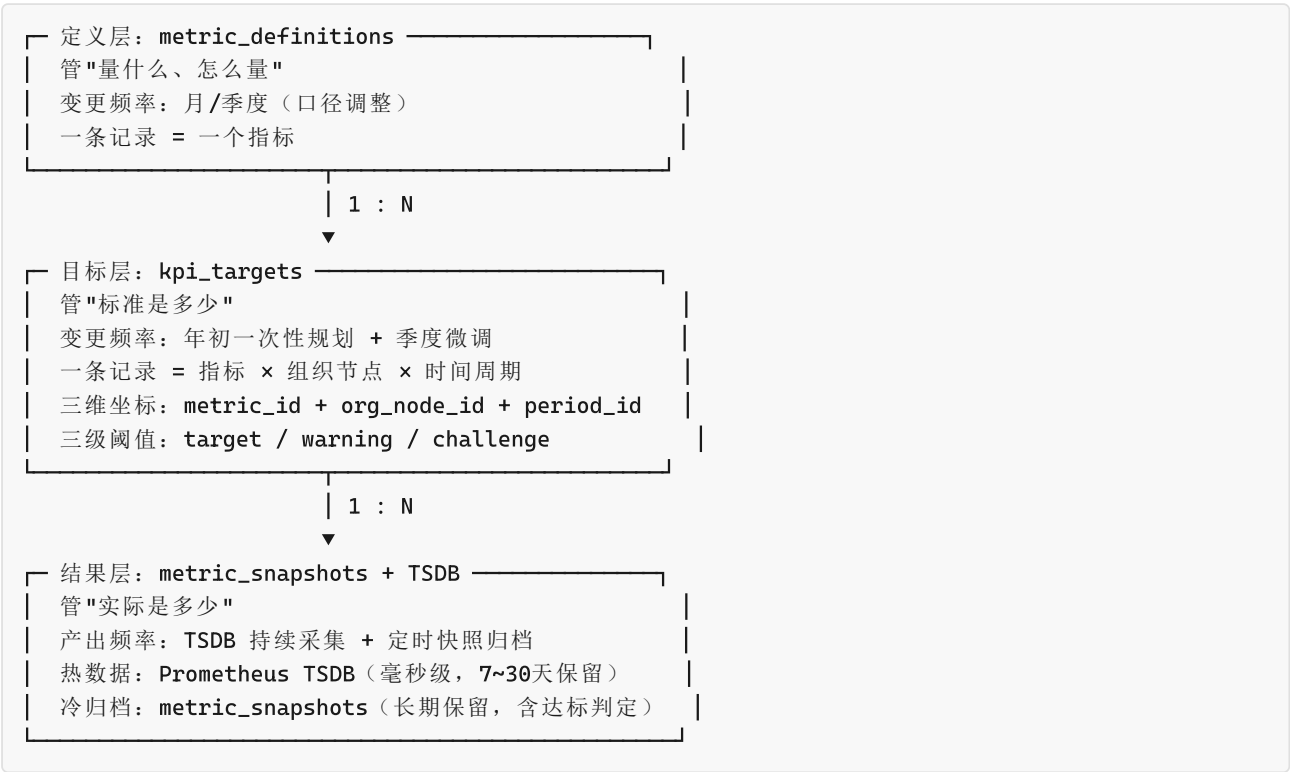
副总师/高层视图：个人 KPI 目标 + 分管部门达标率概览 + 跨部门横向对比图（同一指标，各部门并排对比）。

年初操作流程

从上年复制 → 全局上浮 → 微调公司级网格中的差异项 → 逐个部门审查覆盖 → 副总师/总监岗个性化 → 月度分解确认 → 发布 + 批量导出 PDF 责任书 → 各部门线上签认。全程主管只做"在上年基础上微调差异格"一件事，系统自动处理继承、分解和校验。

二十六、三层数据方案总成

总览



完整 ER 关系

metric_definitions (指标定义) :

```

CREATE TABLE metric_definitions (
  -- 基础标识
  metric_id          STRING PRIMARY KEY,
  metric_name        STRING NOT NULL,
  description         STRING,

  -- 所属（决定"谁的指标、谁能看到"）
  business_domain    STRING NOT NULL,          -- 业务域：支付/物流/技术
  owner_org_node_ids ARRAY<STRING>,             -- 所属组织节点
  owner_team         STRING,                   -- 管理团队
  owner_person       STRING,                   -- 指标负责人

  -- 计算规则
  metric_type        STRING NOT NULL,          -- "gauge" / "counter" / "histogram"
  unit               STRING NOT NULL,
  aggregation        STRING,                   -- "p95" / "avg" / "sum"
  time_window        STRING,                   -- "5m" / "1h"

  -- 数据源与计算模式
  calculation_mode    STRING NOT NULL,         -- "precomputed" / "realtime"
  data_source_type    STRING NOT NULL,         -- "Prometheus" / "ClickHouse"
  query_template      STRING NOT NULL,

  -- 分解策略
  decomposition_strategy STRING DEFAULT 'equal', -- "equal" / "seasonal" / "manual"

  -- 下钻
  drilldown_target    STRING NOT NULL,         -- "business_data" / "statistics"
  drilldown_query     STRING,

  -- 维度
  dimensions          ARRAY<STRING>,           -- ["endpoint", "region"]

  -- 图示
  chart_width         INT DEFAULT 1,           -- 1=1/3, 2=2/3, 3=3/3

  -- 管理
  display_order       INT DEFAULT 0,
  status              STRING DEFAULT 'active', -- "active" / "deprecated"
  created_at          TIMESTAMP,
  updated_at          TIMESTAMP,
  created_by          STRING,
  updated_by          STRING
);

```

所属四字段分工：

字段	回答	用于
<code>business_domain</code>	属于哪个业务域？	按业务域分组、筛选
<code>owner_org_node_ids</code>	哪些组织节点关注？	按组织筛选，决定默认视图
<code>owner_team</code>	哪个团队在管？	跨组织找负责人、告警通知
<code>owner_person</code>	具体谁负责？	指标口径问题的联系人

三层对比

定义层	目标层	结果层	
表 metric_definitions kpi_targets metric_snapshots + TSDB **存什么** 口径、查询模板、维度 阈值 × 组织 × 周期 实测值 + 达标判定 **一条记录** 一个指标 一个指标×组织×周期 一个指标×维度×时间点 **写入频率** 月/季度（口径变更） 年初规划 + 不定期调整 每小时/每天自动 **变更方式** UPDATE（覆盖） INSERT（新增行，旧行归档） INSERT ONLY（不可变） **负责角色** 技术 + 业务协同定义 KPI 管理部门设定 系统自动生成			

查询时三表联动

Stat 面板 → 当前值来自 TSDB 即时查询 + 阈值来自 kpi_targets 折线图 → 近 7 天趋势来自 TSDB 范围查询 + 阈值线叠加 月度达标率 → 来自 metric snapshots (SELECT COUNT WHERE target met=true)

管理界面总图

「 KPI 管理部门 | 年度规划工作台 | 公司级网格编辑 (年+季, 月度折叠) | 部门级差异覆盖 (公司基准并排, 只改差异格) | 岗位级个性化 (继承+覆盖) | 月度分解 (自动展开+手动微调) | 批量操作 (全局上浮、从上年复制、部门间复制) | 导出责任书 PDF |

部门主管 / 领导

目标查看 + 达标追踪

概览卡片 (当前值 + 达标状态 + 目标值)

目标矩阵 (部门目标 vs 公司基准并列)

月度达标趋势图 (叠加部门目标线 + 公司基准线)

导出目标责任书 PDF

一 副总师 / 高层领导 | 跨部门对标视图 | | 一 分管部门
 达标率概览 | | 一 跨部门横向对比图 (同一指标各并排对比) | | 一 个人 KPI 目标 + 达标追踪 |

二十七、列表↔图示切换：指标定义清单 VS 指标目标值清单

核心结论

****看图用指标定义清单，组织/人是筛选条件。管目标分配用指标目标值清单，组织/人是行维度。****

原因：一张图 = 一个指标的时间序列。同一指标不会因为有两个周期的目标值就画两张图——画一张图，叠加两条不同

交互模型

列头筛选模式——和 Excel 一样，每列自带筛选器和排序：

[指标目录](#) | [列表视图](#) | [图示视图](#) | 已筛选：4 个指标 | [业务域=支付 X] [所属组织 $\supseteq$ 支付部 X]

支付总监看到"支付成功率"的图表：叠加线：个人目标($\geq 99.8\%$) + 部门目标($\geq 99.7\%$) + 公司基准($\geq 99.5\%$)

物流总监看到"支付成功率"的图表（同一指标，前提是 owner_org_node_ids 包含物流部）：叠加线：个人目标($\geq 99.5\%$) + 部门目标($\geq 99.5\%$, 继承公司级) + 公司基准($\geq 99.5\%$)

筛选时查询 `owner\_org\_node\_ids` 判断相关性，阈值从 `kpi\_targets` 按岗位→部门→公司优先级查询。

两种清单的适用场景对比

指标定义清单	指标目标值清单
一行是什么	一个指标 × 组织 × 周期
组织/人	筛选条件 行维度
同一指标多行	不会出现 会出现
切换到图示	每指标一张图，阈值线随筛选条件叠加 同一指标可能多张重复图
适合场景	看图（"这批指标趋势如何"） 管目标分配（"这批目标记录对不对"）
典型用户	部门主管/领导日常查看 KPI 主管年初规划、目标调整

实现技术栈

前端框架 React / Vue 表格组件 标准表格库（支持列头筛选 + 多选） 图表渲染 自绘方案 ECharts/Recharts（推荐，轻量可控，无 iframe 开销） → 不建议用 Grafana iframe 嵌入，筛选出 50 个指标 = 50 个 iframe 性能不可接受 数据源 API /api/metrics/:id/data?from=&to= 后端统一封装，屏蔽 PromQL/SQL 差异 /api/metrics/:id/chart-width 宽度设置 /api/metrics?dept=&period=&domain=&type= 指标清单 + 筛选 布局 CSS Grid: grid-template-columns: repeat(3, 1fr); grid-auto-rows: 280px 状态管理 筛选状态 + 宽度设置，简单 React state 即可 性能优化 IntersectionObserver 渐进加载，首屏只渲染可见卡片

二十八、看板定义与看板图示

与指标的三段式（指标定义 → 指标结果 → 指标图示）不同，看板是两段式——看板自身不做计算、不产出中间数据，

术语	对应	说明
看板定义	Dashboard Definition	看板的配置规格——布局、面板组成、变量、数据源绑定、权限。存储在数据库
看板图示	Dashboard	渲染后的视觉效果——用户实际在屏幕上看到的、带叙事结构的指标与非指标内容的聚合

本节讨论这两个概念的内涵，以及看板图示与指标清单图示模式的本质区别。

看板定义

看板 = 把多个指标（及非指标内容）组织在一个固定大小区域中显示的容器。

看板定义是这份容器的配置规格，包含以下要素：

补充 1：固定大小是相对比例。固定的是画布列比例和行高，像素自适应终端。桌面端三列、移动端一列堆叠。

补充 2：看板是统一时间窗口的约束。所有面板共享同一个时间范围和刷新间隔——拖动时间选择器，所有面板同时更新。

补充 3：看板有叙事结构。布局自带诊断路径——自上而下、从宏观到微观。没有叙事结构的看板是一堆数字的拼凑。

补充 4：看板有目标受众。同一指标集面向不同角色产生不同看板（运维看延迟和 CPU，业务看订单量和支付成功率）。

****补充 5：看板定义本身是可配置实体**。**在数据库中对应 ``dashboard_definitions`` 表和 ``dashboard_panels``

****补充 6：看板可包含非指标内容**。**告警列表、部署记录、文本说明、外部链接——这些不来自指标定义表的内容可

图示模式与看板的关系：一场讨论

指标清单的图示模式加上统一时间窗口、全局变量联动、固定 URL 和多配置方案存储之后，技术上与看板几乎无差异，但三件事图示模式做不到——不是因为工程限制，而是因为它们的 ****起点根本不同****。

三条不可约简的结构差异

****差异 1：同一指标在同一视野中出现多次****

图示模式的生成逻辑是 ``metric_definitions`` 中筛选出的每一行 → 一张图，一个指标出现一次。如果允许同一个看板不绑定这个约束。设计师可以把"支付成功率"的当前值放顶部 Stat、趋势放中间折线图、异常明细放底部表格——

****差异 2：非指标内容****

图示模式的数据源是 ``metric_definitions`` 表——每一行都是一个指标。非指标内容（告警列表、部署记录、文本说明）看板的看板的面板可以不绑定任何指标。``dashboard_panels`` 中 ``metric_id`` 可以为空——这个面板只是一个文本块、一个

****差异 3：跨指标复合运算****

图示模式每张图对应一条 ``query_template``、一个 ``metric_id``。"支付成功率偏差"不是一个独立的指标——它是两个指标。看板可以在面板级别做跨指标运算——取指标 A 的值减去指标 B 的值，在面板上画出来，而不需要把计算结果定义为

三条差异的同源根因

图示模式	看板	
创作主体	系统（自动映射）	人（手工编排）
生成规则	一行指标 = 一张图	无固定规则，设计师决定
同一指标多次出现	不能（一对一约束）	能（设计师决策）
非指标内容	不能（数据源是指标表）	能（面板不强制绑定指标）
跨指标运算	不能（每图一查询）	能（面板级组合多指标）

****图示模式是"从指标到图示"的自动化流水线，看板是"从判断到表达"的人工创作。**** 三条差异不是偶然的工程限制

看板解决的根本问题

看板解决的是"信任"问题，不是"查看"问题。

图示模式是一个工具。用户需要知道自己想看什么——筛选哪个业务域、哪个组织、哪个周期。用户是主动的探索者，在

看板是一个承诺。用户不需要知道里面有什么指标、怎么筛选、怎么排列。用户是被服务的消费者，来拿答案。

设计一个看板时，设计师做的每一个决定背后都是一个判断：34 个指标中只放 4 个（其余 30 个被有意排除）、这

****图示模式 = 指标的集合（collection），以指标为起点。看板 = 判断的产物（curated artifact），以受众为起**

两种创建路径，同一个产物

两者最终产生的是同一个东西——一份**看板定义**（`dashboard\_definitions` + `dashboard\_panels`），渲染为

路径 1：从指标出发 → 筛选 → 自动生成网格 → 保存为看板定义 → 渲染为看板图示 路径 2：从空白画布出发 → 手动拖入面板 → 编排布局 → 保存为看板定义 → 渲染为看板图示

产物相同：一份看板定义 + 对应的看板图示

路径 1 是"筛选出了一个有价值的指标组合，固化为看板定义"。路径 2 是"心里有一个受众画像和诊断路径，手工创

二十九、看板清单与看板图示

模式上移：从指标清单到看板清单

指标清单→指标图示的模式可以上移一层，应用于看板本身：

指标层：metric_definitions 表 → 筛选/排序 → 切换图示 → 每指标一张图 看板层：dashboard_definitions 表 → 筛选/排序 → 切换图示 → 每看板一幅图

看板定义的所属字段

与 `metric\_definitions` 对称，`dashboard\_definitions` 增加所属字段：

```
```sql
-- dashboard_definitions 已有字段 + 新增所属字段
business_domain STRING NOT NULL, -- 业务域
owner_org_node_ids ARRAY<STRING>, -- 所属组织节点
owner_team STRING, -- 管理团队
owner_person STRING, -- 负责人
visible_to_org_nodes ARRAY<STRING>, -- 可见范围（与 owner 不同：owner 决定"归谁管"，visible 决定"谁能看"）
```

## 看板定义清单

看板目录

[列表视图] [图示视图]

已筛选：5 个看板 | [所属组织 ⊇ 支付部 ×]

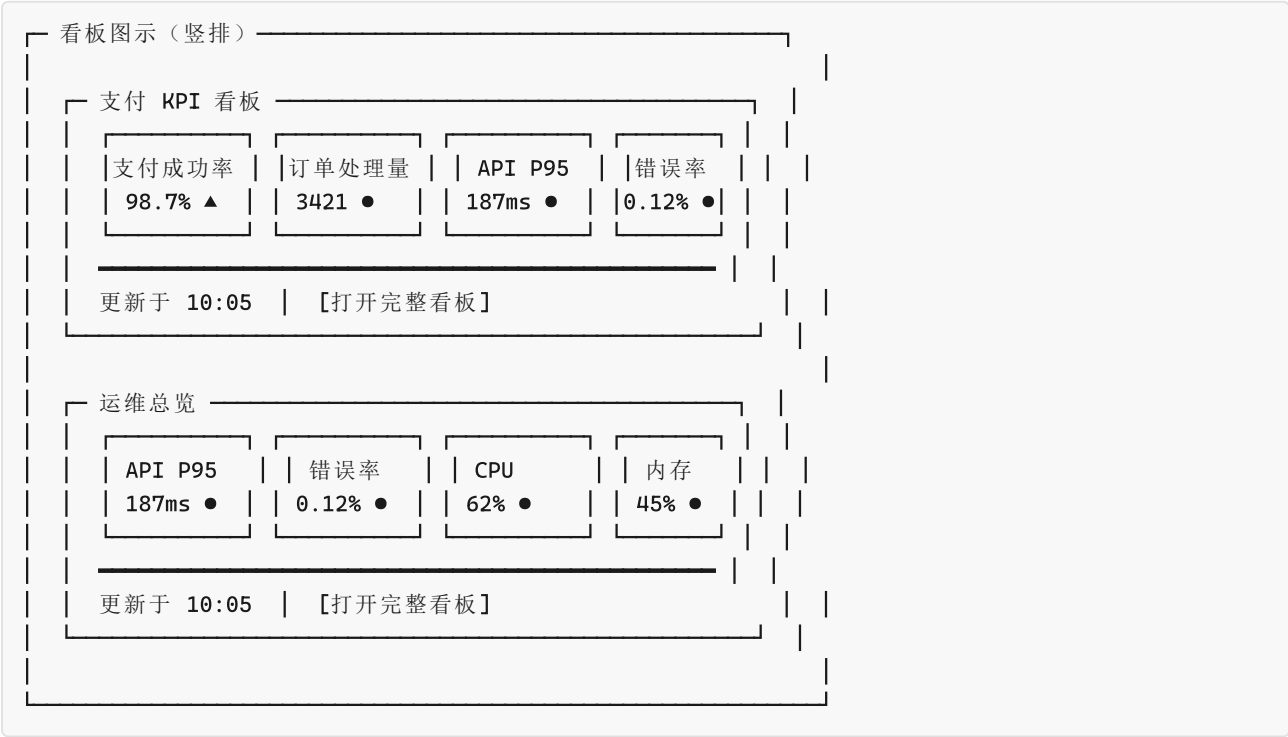
看板名称	业务域	所属组织	受众	指标数	更新	创建人	状态		
▼▼	▼▼	▼▼	▼▼	▼	▼	▼	▼▼		
支付KPI	支付	支付部	主管	4	30s	张三	活跃		
运维总览	技术	支付部	运维	8	15s	李四	活跃		
物流KPI	物流	物流部	主管	3	30s	王五	活跃		
技术质量	技术	技术部	管理	6	60s	赵六	草稿		
公司KPI	全公司	全公司	高层	12	60s	KPI组	活跃		

\* 支付总监登录后默认筛选：所属组织 ⊇ 支付部/支付总监，看到 2 个看板

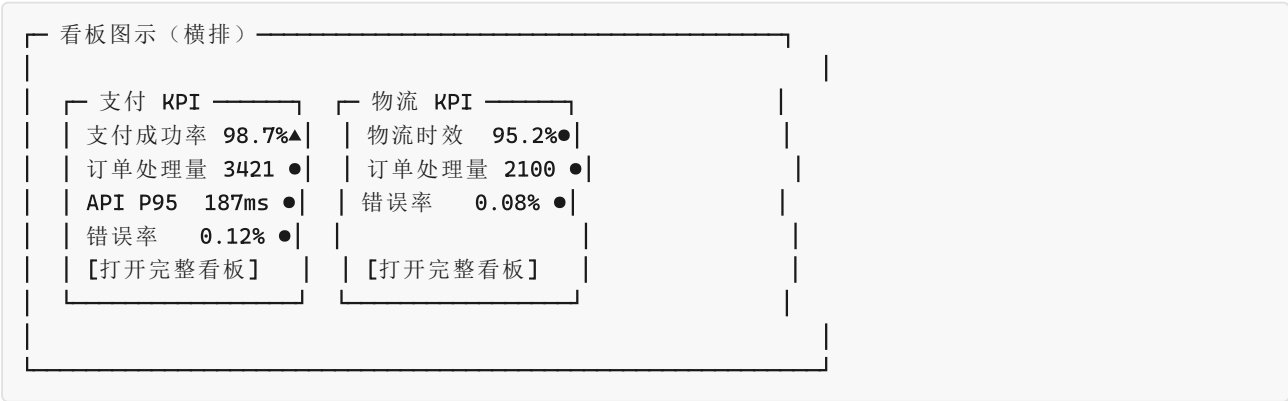
切换到图示视图：竖排与横排

图示视图中的每个看板以**摘要卡片**形式展示——只渲染该看板第一行的概览面板（通常是 Stat 面板阵列），不渲染完整看板。完整看板通过"打开完整看板"链接跳转。

**竖排模式**——上下滚动浏览：



**横排模式**——并排对比：



横排模式的核心价值是**并排对比**——支付总监一眼看到支付部和物流部的 KPI 并排，无需上下滚动对照。

不同人员看到不同内容

与指标清单相同的机制——用户登录后，看板清单根据其组织归属自动筛选：



技术副总登录：

- 所属组织  $\supseteq$  技术部 或 技术副总
- 看到 3 个看板（技术质量 + 运维总览 + 公司 KPI）

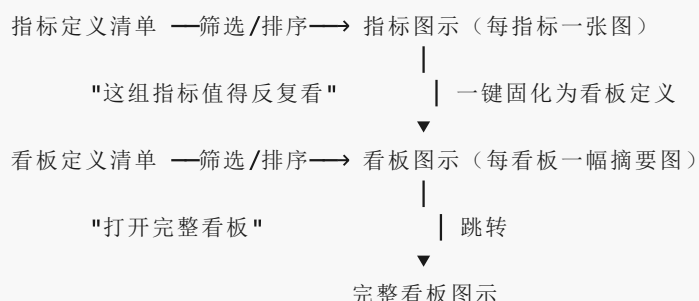
## 适用场景

**高管晨检：**技术副总打开看板清单，筛选"技术部+支付部"，横排对比三个部门的 KPI 看板摘要，发现支付成功率红了——点进支付 KPI 细看。

**KPI 主管月度审查：**月初打开看板清单，筛选"所有部门 KPI 看板"，竖排滚动浏览各看板摘要，发现技术部门抬头——点进去展开。

**新员工入职：**不知道有什么看板。打开看板清单，默认筛选已经限定为"自己部门的看板"，浏览后理解团队日常关注什么。

## 与指标图示模式的关系



两条清单→图示链，上下衔接。从指标探索到看板固化，再从看板清单审视到完整看板深入——形成完整的导航层次。

## 三十、看板定义驱动的技术方案

与指标定义驱动计算链条不同，看板定义驱动的是**分发链条**——同一份看板部署到 Web、PDF 报表、大屏、邮件等多个渠道。

## 数据模型

**dashboard\_definitions:**

```
CREATE TABLE dashboard_definitions (
 dashboard_id STRING PRIMARY KEY,
 dashboard_name STRING NOT NULL,
 description STRING,

 -- 所属
 business_domain STRING NOT NULL,
 owner_org_node_ids ARRAY<STRING>,
 owner_team STRING,
 owner_person STRING,
 visible_to_org_nodes ARRAY<STRING>,

 -- 展示
 default_time_range STRING DEFAULT '1h',
```

```

auto_refresh STRING DEFAULT '30s',
display_mode STRING DEFAULT 'normal', -- "normal" / "kiosk" / "embed"

-- 变量
variables JSON,

-- 分发
schedule STRING, -- "0 9 * * 1" 或 NULL
report_recipients ARRAY<STRING>,

-- 管理
status STRING DEFAULT 'active',
created_at TIMESTAMP,
updated_at TIMESTAMP,
created_by STRING,
updated_by STRING
);

```

## dashboard\_panels:

```

CREATE TABLE dashboard_panels (
 panel_id STRING PRIMARY KEY,
 dashboard_id STRING NOT NULL, -- FK → dashboard_definitions

 -- 指标绑定 (NULL = 非指标面板)
 metric_id STRING, -- FK → metric_definitions

 -- 非指标面板内容
 panel_type STRING DEFAULT 'auto', -- "auto" / "text" / "image" / "iframe"
 panel_content STRING,

 -- 布局
 row_position INT NOT NULL,
 col_position INT NOT NULL,
 col_span INT DEFAULT 1,

 -- 覆盖
 override_title STRING,
 panel_query STRING, -- 覆盖 metric 的 query_template (跨指标运算)

 -- 排序
 sort_order INT DEFAULT 0
);

```

## 数据示例

```

-- 支付 KPI 看板定义
INSERT INTO dashboard_definitions VALUES (
 'kpi_payment', '支付 KPI 看板', '支付部门日常 KPI 监控',
 '支付', ARRAY['dept_payment', 'role_dir_pay'], '支付团队', 'zhangsan',
 ARRAY['dept_payment', 'role_dir_pay', 'role_vp_tech'],
 '7d', '60s', 'normal',
 '{"name": "region", "type": "query", "query": "SELECT DISTINCT region FROM orders"}',
 '0 9 * * 1', ARRAY['zhangsan@corp.com', 'vp@corp.com'],
 'active', NOW(), NOW(), 'zhangsan', 'zhangsan'
);

```

```
-- 面板定义
INSERT INTO dashboard_panels VALUES
-- 第 1 行: 概览卡片 (4 个 Stat)
('p1', 'kpi_payment', 'payment_success_rate', 'auto', NULL, 0, 0, 1, NULL, NULL, 1),
('p2', 'kpi_payment', 'order_throughput', 'auto', NULL, 0, 1, 1, NULL, NULL, 2),
('p3', 'kpi_payment', 'api_latency_p95', 'auto', NULL, 0, 2, 1, NULL, NULL, 3),
('p4', 'kpi_payment', 'error_rate', 'auto', NULL, 0, 3, 1, NULL, NULL, 4),
-- 第 2 行: 趋势图 (3 个 TimeSeries)
('p5', 'kpi_payment', 'payment_success_rate', 'auto', NULL, 1, 0, 1, NULL, NULL, 5),
('p6', 'kpi_payment', 'order_throughput', 'auto', NULL, 1, 1, 1, NULL, NULL, 6),
('p7', 'kpi_payment', 'api_latency_p95', 'auto', NULL, 1, 2, 1, NULL, NULL, 7),
-- 第 3 行: 非指标面板 (文本说明)
('p8', 'kpi_payment', NULL, 'text', '## 关注事项\n- Q2 目标收紧至 99.7%', 2, 0, 3, '注意事项', NULL)
```

注意: `payment_success_rate` 出现了两次 (p1 Stat + p5 TimeSeries)。这是看板特征——同一指标多次出现、不同用途, 图示模式做不到这一点。

## CI/CD 生成流水线

```
dashboard_definitions + dashboard_panels
|
| 变更触发: INSERT / UPDATE / DELETE
|
▼
CI/CD Pipeline
|
| ① 读取全量看板定义
| SELECT d.*, p.*
| FROM dashboard_definitions d
| JOIN dashboard_panels p ON d.dashboard_id = p.dashboard_id
| LEFT JOIN metric_definitions m ON p.metric_id = m.metric_id
| WHERE d.status = 'active'
|
| ② 按 dashboard_id 分组生成各产物
| for each dashboard:
| generate_grafana_json(dashboard)
| generate_catalog_entry(dashboard)
| if dashboard.schedule:
| register_snapshot_job(dashboard)
| sync_permissions(dashboard)
|
| ③ 部署
| — Grafana JSON → Provisioning API
| — 看板目录 → 静态 HTML / API
| — 定时快照 → K8s CronJob
| — 权限 → Grafana Teams/Folders
```

## 核心生成逻辑

```
def generate_grafana_json(dashboard):
 """从看板定义 + 面板定义生成 Grafana Dashboard JSON"""
 panels = db.query("""
 SELECT p.*,
 m.metric_name, m.unit, m.query_template,
 m.data_source_name, m.chart_width,
 m.metric_type, m.drilldown_target
 FROM dashboard_panels p
 LEFT JOIN metric_definitions m ON p.metric_id = m.metric_id
 """)
```

```

WHERE p.dashboard_id = ?
ORDER BY p.row_position, p.col_position
""", dashboard.dashboard_id)

rows = group_by(panels, 'row_position')
panel_list = []
pid = 1

for row_idx, row_panels in rows.items():
 col_offset = 0
 for p in row_panels:
 # Grafana 24 列网格: 1/3=8, 2/3=16, 3/3=24
 w = int(24 * p.col_span / 3)
 x = int(24 * col_offset / 3)
 h = 6

 if p.metric_id is None:
 panel = {
 "id": pid, "title": p.override_title,
 "type": "text",
 "gridPos": {"x": x, "y": row_idx * 6, "w": w, "h": h},
 "options": {"mode": "markdown", "content": p.panel_content}
 }
 else:
 metric = p.metric
 panel = {
 "id": pid,
 "title": p.override_title or metric.metric_name,
 "type": "stat" if w ≤ 8 else "timeseries",
 "datasource": {"type": "prometheus", "uid": metric.data_source_name},
 "targets": [{"expr": p.panel_query or metric.query_template, "refId": "A"}],
 "gridPos": {"x": x, "y": row_idx * 6, "w": w, "h": h},
 "fieldConfig": {
 "defaults": {
 "unit": metric.unit,
 "thresholds": {"mode": "absolute", "steps": [
 {"color": "green", "value": None},
 {"color": "red", "value": 80}
]}
 }
 }
 }

 panel_list.append(panel)
 pid += 1
 col_offset += p.col_span

return {
 "title": dashboard.dashboard_name,
 "description": dashboard.description,
 "refresh": dashboard.auto_refresh,
 "time": {"from": f"now-{dashboard.default_time_range}", "to": "now"},
 "templating": {"list": dashboard.variables or []},
 "panels": panel_list,
 "tags": [dashboard.business_domain, dashboard.owner_team]
}

```



## 定时快照任务注册

```
def register_snapshot_job/dashboard):
 """为有 schedule 的看板注册 CronJob"""
 if not dashboard.schedule:
 return

 job = {
 "apiVersion": "batch/v1",
 "kind": "CronJob",
 "metadata": {"name": f"dashboard-snapshot-{dashboard.dashboard_id}"},
 "spec": {
 "schedule": dashboard.schedule,
 "jobTemplate": {
 "spec": {
 "template": {
 "spec": {
 "containers": [{
 "name": "snapshot",
 "image": "grafana-snapshot:latest",
 "args": [
 "--dashboard-id", dashboard.dashboard_id,
 "--recipients", ",".join(dashboard.report_recipients)
],
 "env": [
 {"name": "GRAFANA_URL", "value": os.getenv("GRAFANA_URL")},
 {"name": "GRAFANA_TOKEN", "valueFrom": {
 "secretKeyRef": {"name": "grafana-token", "key": "token"}
 }}
]
 }
],
 },
 "restartPolicy": "OnFailure"
 }
 }
 }
 }

 kubernetes.client.BatchV1Api().create_namespaced_cron_job("monitoring", job)
```

## 一键固化：从指标图示到看板定义

```
def save_as_dashboard(user_id, filter_state, selected_metrics):
 """用户在指标图示模式中筛选出指标 → 一键固化为看板定义"""

 dashboard_id = generate_id()
 db.insert("dashboard_definitions", {
 "dashboard_id": dashboard_id,
 "dashboard_name": f"我的看板 {now().strftime('%Y%m%d-%H%M')}",
 "business_domain": user_default_domain(user_id),
 "owner_org_node_ids": [user_org_node(user_id)],
 "owner_team": user_team(user_id),
 "owner_person": user_id,
 "visible_to_org_nodes": [user_org_node(user_id)],
 "default_time_range": filter_state.time_range or "1h",
 "auto_refresh": "30s",
```

```

 "created_by": user_id,
 "status": "active"
 })

 for i, metric_id in enumerate(selected_metrics):
 metric = db.query(
 "SELECT chart_width FROM metric_definitions WHERE metric_id = ?", metric_id)
 row, col = auto_layout(selected_metrics, i)
 db.insert("dashboard_panels", {
 "panel_id": generate_id(),
 "dashboard_id": dashboard_id,
 "metric_id": metric_id,
 "panel_type": "auto",
 "row_position": row,
 "col_position": col,
 "col_span": metric.chart_width,
 "sort_order": i
 })

 trigger_pipeline("dashboard_deploy", {"dashboard_id": dashboard_id})
 return f"/dashboards/{dashboard_id}"

```

## 自动化产物对比

	指标定义驱动	看板定义驱动
本质	计算链条（同一查询部署到 Recording Rule + 面板 + 告警）	分发链条（同一视图部署到 Web + PDF + 大屏 + 邮件）
主产物	Recording Rule + Grafana 面板 + 告警规则	Grafana 看板 JSON
次级产物	指标目录	看板目录 + 定时报表 + 大屏/嵌入 URL + 权限配置
变更触发	口径变更 → 更新查询模板 → 全部下游同步	看板布局变更 → 更新 JSON → 所有分发渠道同步

## 全链路自动化总图

```

metric_definitions（一行）
|
├── Recording Rules
├── Grafana 面板
├── 告警规则
└── 指标目录
 |
 | 在指标图示模式中筛选/观察
 | 一键固化为看板定义
 ▼
dashboard_definitions（一行）+ dashboard_panels（N 行）
|
├── Grafana 看板 JSON
├── 看板目录
├── 定时报表 CronJob
├── 大屏/嵌入 URL
└── 权限同步

```

技术栈：

- └─ 数据库：PostgreSQL / MySQL
- └─ CI/CD：GitHub Actions / GitLab CI / Jenkins
- └─ 脚本：Python（读取 DB + 生成 JSON + 注册 CronJob）
- └─ 部署：kubectl + Grafana Provisioning API
- └─ 渲染：自研 ECharts/Recharts 前端（看板目录 + 图示模式）

## 三十一、封装为 91 规范引擎：指标引擎与看板引擎

根据 91 规范（91-eos-biz-eng-spec.md v5.6），EOS 平台已有 12 个引擎。其中**指标引擎（@engine-indicator）**和**看板引擎（@engine-kanban）**均已在列，归属功能引擎层（EL2）• 业务管理类。本节讨论的不是“新建”引擎，而是基于本次讨论的方案对两个引擎的配置单元（CU）增强。

### 引擎在 91 规范架构中的位置

EL 三层引擎生成链：

- EL3 实体引擎 —生成—> 物理表（业务数据源）
- EL2 功能引擎 —生成—> 功能表单 / 主子表单 / 指标页面 / 看板页面
- EL1 菜单引擎 —生成—> 菜单树 + 场景业务

功能引擎（EL2）结构：

- └─ 共享能力（表单能力 + 功能能力 + 组装能力 + 权限能力）
- └─ 各具体功能引擎 |
  - └─ 产品数据类：流程引擎、工单引擎、台账引擎
  - └─ 业务管理类：指标引擎、看板引擎、项目引擎、WBS引擎、计划引擎、进展引擎、定时任务引擎

每个具体功能引擎共享表单能力、权限能力与组装能力，各自定义专属的功能 CU。

### 指标引擎（@engine-indicator）CU 增强方案

当前 CU：指标定义管理、2x3 网格布局配置

增强后的 CU 清单：

CU ID	CU 名称	状态	说明
@indicator-cu-definition	指标定义管理	已有·增强	新增所属四字段（business_domain / owner_org_node_ids / owner_team / owner_person）、chart_width、decomposition_strategy
@indicator-cu-grid-layout	指标图示网格	已有·增强	列头筛选/排序（Excel 模式）、图示宽度三选一、统一时间窗口+全局变量联动、列表↔图示标签页切换
@indicator-cu-threshold	目标值管理	新增	kpi_targets 三维立方体（组织×时间×指标）、继承分解规则、年度规划工作台、阈值变更审计、审批流
@indicator-cu-computation	指标计算与存储	新增	预计算（Prometheus TSDB 采集聚合）、即时计算（ClickHouse 明细查询）、定时快照（metric_snapshots）、分层存储（热/温/冷）
@indicator-cu-drilldown	指标下钻	新增	聚合值→业务数据明细 / 统计结果、下钻终点元信息标注（业务数据 / 统计结果 / 外部链路 / 无下钻）

CU ID	CU 名称	状态	说明
@indicator-cu-organization	组织维度管理	新增	org_nodes 组织树 + time_periods 时间树、按角色默认筛选、多级阈值线叠加（岗位→部门→公司）

### 增强后的 IPO：

<b>I（输入）：</b>       └─ 各引擎运行产生的原始数据（实体引擎的实体变更日志、流程引擎的流程执行记录、工单引擎的工单处理记录.....）     └─ 业务域对指标定义的需求（"量什么、怎么量"）     └─ KPI 管理部门的目标值规划（"标准是多少"）     └─ 看板引擎的指标查询请求			
<b>P（处理）：</b>       └─ [已有] 指标定义 CRUD + 2x3 网格布局     └─ [增强] 四字段所属管理 + 列头筛选/排序     └─ [新增] 目标值三维管理 + 年度规划 + 继承分解     └─ [新增] 预计算采集聚合 + 即时计算查询 + 定时快照     └─ [新增] 下钻链路（聚合值→明细）     └─ [新增] 组织维度默认筛选 + 多级阈值线叠加			
<b>O（输出）：</b>       └─ 指标定义数据（metric_definitions 表）     └─ 目标值数据（kpi_targets 表）     └─ 指标时间序列（Prometheus TSDB）     └─ 指标快照 + 达标判定（metric_snapshots 表）     └─ 指标图示 + 指标数据 API → 供看板引擎消费			

## 看板引擎 (@engine-kanban) CU 增强方案

当前 CU：看板布局配置、视图泳道配置

增强后的 CU 清单：

CU ID	CU 名称	状态	说明
@kanban-cu-layout	看板布局编排	已有·增强	支持非指标面板（文本/图片/iframe）、同一指标多次出现、跨指标复合运算面板、叙事结构布局（宏观→微观）
@kanban-cu-swimlane	视图泳道与图示模式	已有·增强	竖排/横排两种看板清单图示模式、看板摘要卡片（仅渲染第一行概览面板）
@kanban-cu-definition	看板定义管理	新增	dashboard_definitions + dashboard_panels 表、所属四字段、受众权限（visible_to_org_nodes）、变量绑定、时间窗口预设
@kanban-cu-distribution	看板分发	新增	CI/CD 自动生成 Grafana Dashboard JSON、定时快照 CronJob（PDF/PNG 邮件报表）、大屏/嵌入 URL、权限同步（Grafana Teams/Folders）
@kanban-cu-catalog	看板清单	新增	列头筛选/排序（Excel 模式）、看板清单↔图示标签页、按角色默认筛选、跨看板并排对比
@kanban-cu-solidify	一键固化	新增	从指标引擎的指标图示模式一键保存为看板定义（筛选条件→变量、指标列表→面板、布局→自动排版）

增强后的 IPO：

I（输入）：

- 指标引擎的指标定义 + 指标数据 API（当前值 + 趋势）
- 看板编排需求（受众、布局、叙事结构）
- 非指标内容（告警列表、部署记录、文本说明、外部链接）
- 分发配置（定时快照调度、邮件接收人）

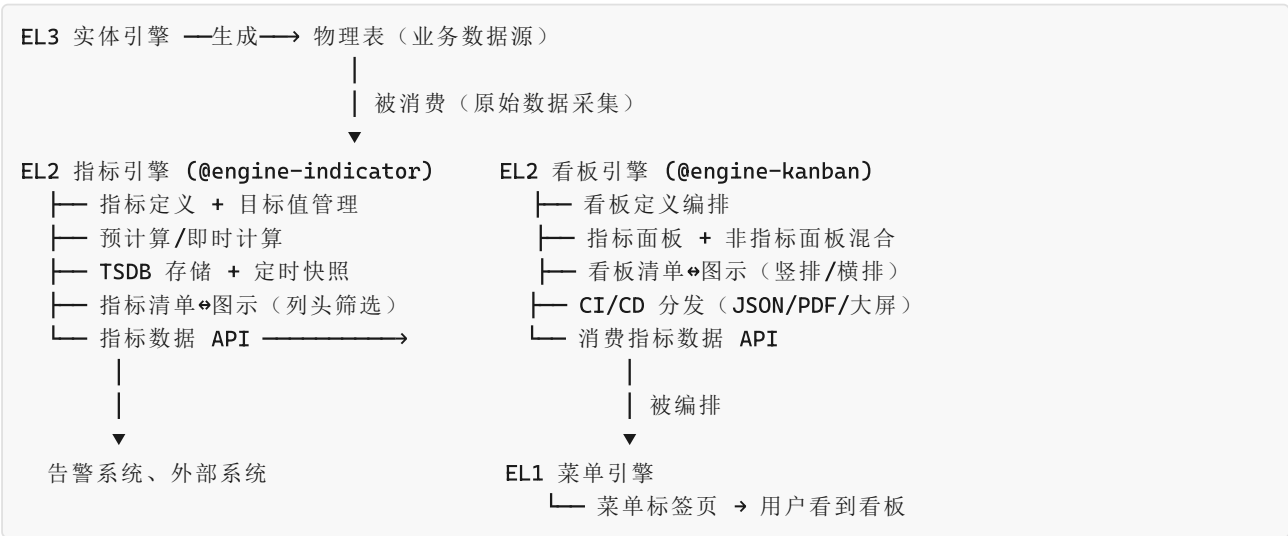
P（处理）：

- 【已有】看板布局编排 + 视图泳道配置
- 【增强】非指标面板混合编排 + 同一指标多次出现 + 跨指标运算
- 【新增】看板定义管理（手动编排 / 从指标图示一键固化）
- 【新增】CI/CD: dashboard\_definitions → Grafana JSON + CronJob + 权限
- 【新增】看板清单+图示 + 竖排/横排切换

O（输出）：

- 看板定义数据（dashboard\_definitions + dashboard\_panels 表）
- 看板图示（Grafana Dashboard / 自研前端渲染）
- 看板清单 + 图示（竖排/横排）
- 定时快照 / 邮件报表
- 大屏 / 嵌入 URL
- 权限绑定（Grafana Teams/Folders）

两引擎在 EOS 架构中的关系



**指标引擎 = EOS 的度量基础设施**（数据采集→计算→存储→API 输出） **看板引擎 = EOS 的信息分发基础设施**（编排→生成→渲染→快照分发）

两者的关系与实体引擎→表单引擎的关系同构——数据生产者与数据编排消费者。

与 91 规范 CU 设计规则的对应

91 规范规定：“CU 是引擎提供的配置能力单元，其能力边界、配置入口、配置要素、系统处理、运行期输出和 FR-ENG 性能指标仅由 eng 路径设计。”

本方案中的所有新增 CU 均需由 **eng 路径**（wft02-eng → wft03-eng → wft05-eng）完成正式设计。各 CU 的配置入口、配置要素、系统处理逻辑由指标引擎和看板引擎的 eng 路径任务定义。

本讨论各章节与引擎 CU 的对应

文档章节	讨论内容	对应引擎 CU
一~十二	指标定义/结果/图示、 name/label/Stat/PromQL/TSDB	@indicator-cu-definition + @indicator-cu-computation
十三~二十一	预计算 vs 即时计算、下钻、列存压缩、物化视图	@indicator-cu-computation + @indicator-cu-drilldown
二十二	区分预计算/即时计算、下钻终点元信息	@indicator-cu-drilldown
二十三	阈值表 + 快照表 + 分层存储	@indicator-cu-threshold + @indicator-cu-computation
二十四~二十五	阈值管理界面 + 年度 KPI 规划	@indicator-cu-threshold + @indicator-cu-organization
二十六	三层数据方案（定义/目标/结果）	@indicator-cu-definition + @indicator-cu-threshold + @indicator-cu-computation
二十七	列表↔图示切换 + 所属字段筛选	@indicator-cu-grid-layout + @indicator-cu-organization
二十八	看板定义 vs 指标图示模式（三条不可约简差异）	@kanban-cu-definition + @kanban-cu-layout + @kanban-cu-solidify
二十九	看板清单↔图示	@kanban-cu-catalog + @kanban-cu-swimlane
三十	看板定义驱动技术方案	@kanban-cu-distribution + @kanban-cu-definition